

E0306 TALLER DE PROYECTO I

Informe final

Médium: dispositivo para captar y transmitir señalesGrupo 5Majoros, Lorenzo¹ (03296/1)Nihany, Santiago¹ (03012/3)Triviño Loyola, Gastón Eduardo¹ (02997/2)Seery, Juan Martín^{1*} (03471/9)

09/02/2026

¹{lorenzomajoros,santiago.nihany,gastontrivino,juan.seery}@alu.ing.unlp.edu.ar

*Autor responsable del informe

Índice

1. Introducción	3
2. Objetivos del proyecto	3
2.1. Objetivos Primarios	3
2.2. Objetivos secundarios	3
3. Análisis de requerimientos	4
3.1. Requerimientos de los periféricos (funcionales)	4
3.2. Requerimientos del sistema (funcionales)	4
3.3. Requerimientos del proyecto (no funcionales)	4
4. Diseño de hardware	5
4.1. Diseño de Esquemático	5
4.1.1. Teclas (<i>entrada</i>)	5
4.1.2. Pantalla OLED (<i>salida</i>)	5
4.1.3. Módulo emisor/receptor infrarrojo (<i>entrada/salida</i>)	6
4.1.4. Módulo emisor/receptor de radiofrecuencia (<i>entrada/salida</i>)	7
4.1.5. Memoria microSD (<i>entrada/salida</i>)	7
4.1.5.1. Modificaciones	8
4.1.6. Alimentación	8
4.1.7. Resumen	9
4.2. Diseño del circuito impreso	9
4.2.1. Proceso de diseño del PCB	9
5. Diseño del firmware	11
5.1. Arquitectura general	11
5.1.1. Diagrama de flujo	11
5.1.2. Bibliotecas externas y dependencias	12
5.2. Módulos desarrollados	12
5.2.1. Sistema RTOS	12
5.2.1.1. Primitivas de RTOS creadas	12
5.2.1.2. Pruebas	12
5.2.2. Pantalla OLED	13
5.2.3. MicroSD	13
5.2.4. Módulo IR	14
5.2.4.1. NEC	14
5.2.4.2. Software	15
5.2.4.2.1. Adquisición de señal infrarroja (RX)	15

5.2.4.2.2.	Procesamiento y decodificación del protocolo NEC	16
5.2.4.2.3.	Generación de señal infrarroja (TX – Emisión NEC)	16
5.2.4.2.4.	Generación temporal	16
5.2.4.2.5.	Programa Principal	16
6.	Ensayos y mediciones	17
6.1.	Módulo de infrarrojos (IR)	17
6.1.1.	Funcionamiento correcto de transmisión y recepción	17
6.1.2.	Implementación alternativa con interrupciones y descarte de la solución	19
6.1.3.	Alimentación	19
6.2.	Módulo de radiofrecuencia (RF)	20
6.2.1.	Validación mínima de funcionamiento	20
6.2.2.	Primer ensayo: captura de señal de 315 MHz	20
6.2.3.	Segundo ensayo: captura de señal de 433,92 MHz	21
6.2.4.	Tercer ensayo: ataque de interferencia (<i>jamming</i>)	23
6.2.5.	Notas adicionales de los ensayos	23
6.3.	Ensayos de integración	24
7.	Conclusiones	26
7.1.	Cumplimiento	26
7.2.	División de tareas	27
7.3.	Conclusiones generales	28
	Bibliografía	28
A.	Apéndices	29
A.1.	Lista de materiales	29
A.2.	Presupuesto	30
A.3.	Circuitos y esquemáticos	31
A.4.	PCB	33
A.5.	Manual de usuario	34
A.5.1.	Controles del equipo	34
A.5.2.	Organización de archivos en microSD	34
A.5.3.	Flujo de uso en el dispositivo	34
A.5.3.1.	Grabar una señal IR	34
A.5.3.2.	Grabar una señal RF (CC1101)	34
A.5.3.3.	Reproducir una señal	34
A.5.3.4.	Borrar una señal	34
A.5.4.	Recomendaciones de uso	34
A.5.5.	Scripts de PC para desarrollo y análisis	35
A.5.5.1.	Preparación del entorno	35
A.5.5.2.	Terminal serie (<code>terminal.py</code>)	35
A.5.5.3.	Inspección de archivos <code>.sig</code> (<code>signals.py</code>)	35
A.5.6.	Solución rápida de problemas	35
A.6.	Formato de archivo <code>.sig</code>	36
A.6.1.	Estructura general	36
A.6.1.1.	Header (32 bytes)	36
A.6.1.2.	Codificación de duraciones (<i>edges</i>)	36
A.6.1.3.	Metadata opcional (TLV)	36
A.6.1.4.	Verificación de integridad	36
A.6.2.	Script de inspección	37

1. Introducción

En los últimos años, la interacción entre dispositivos electrónicos y el entorno cotidiano se ha visto fuertemente influenciada por tecnologías de comunicación inalámbrica de bajo costo y corto alcance — mandos a distancia, sensores de movimiento, sistemas de alarma, controles de portones eléctricos y equipos de domótica son solo algunos ejemplos de dispositivos que utilizan protocolos simples basados en señales infrarrojas (IR) y radiofrecuencia (RF) en bandas ISM/SRD¹ para transmitir información de manera eficiente y económica. Estas tecnologías, aunque ampliamente difundidas, suelen ser percibidas por el usuario final como cajas negras: se utilizan sin comprender cómo se generan, cómo se codifican los datos ni cómo podrían integrarse en nuevos desarrollos.

El proyecto, llamado *Médium*, tiene como motivación principal acercar estas tecnologías al ámbito educativo, brindando una herramienta didáctica y práctica que permita captar, analizar, almacenar y replicar señales IR y RF — un instrumento que le permita al usuario interactuar directamente con los distintos aparatos que tenga en su casa. También podría llegar a ser utilizado para realizar depuraciones no profesionales, como detectar si la comunicación entre dos dispositivos funciona correctamente.

El dispositivo propuesto integrará módulos de recepción y transmisión (para IR y RF), una unidad de almacenamiento local para registrar capturas y una interfaz física básica (pantalla y controles) que permita al usuario interactuar con las señales. Por ejemplo, este dispositivo podría ser utilizado para capturar y almacenar la señal de un control remoto de televisión. A partir de la señal guardada, el proyecto permitiría reproducirla con el mismo dispositivo para encender, apagar o cambiar de canal, replicando así la función original del control sin necesidad de contar físicamente con él.

El proyecto es innovador porque se ubica en un espacio intermedio entre los analizadores profesionales de protocolos (costosos y especializados) y las soluciones caseras que suelen limitarse a un único protocolo. El único caso de un dispositivo similar de venta libre es Flipper Zero [1]. Una herramienta compacta y portátil que integra múltiples capacidades: lectura y emulación de etiquetas RFID/NFC, recepción y transmisión en bandas RF, captura y reproducción de señales infrarrojas, interfaces GPIO para experimentar con periféricos y una interfaz local con pantalla y botones que facilita su uso en campo. Flipper Zero ha mostrado la utilidad de consolidar en un solo equipo funciones que antes requerían múltiples dispositivos y conocimientos especializados, y ha generado una comunidad activa que desarrolla firmware, librerías y casos de uso.

En definitiva, la relevancia del proyecto radica en ofrecer una plataforma flexible y accesible para experimentar con tecnologías de comunicación de uso cotidiano, acercando a estudiantes, docentes y entusiastas al entendimiento de protocolos de transmisión inalámbrica básicos, y sentando las bases para aplicaciones futuras en domótica, control remoto y sistemas de seguridad.

2. Objetivos del proyecto

Con el fin de establecer con claridad las metas a alcanzar y organizar de manera eficiente el trabajo, se han delimitado los objetivos específicos del proyecto, diferenciando entre aquellos de cumplimiento obligatorio (primarios) y aquellos que constituyen posibles mejoras o ampliaciones (secundarios).

2.1. Objetivos Primarios

- Detectar y muestrear señales infrarrojas (IR).
- Detectar y muestrear señales de radiofrecuencia (RF).
- Brindar una interfaz de usuario para controlar el dispositivo (elegir captura, guardar, entre otras).
- Perdurar las señales captadas.

2.2. Objetivos secundarios

- Interpretar protocolos comunes de IR.
- Interpretar protocolos comunes de RF.
- Visualizar las señales en el mismo dispositivo.

¹Los *Short-range devices* son dispositivos de baja potencia con poco rango de transmisión. De todas las bandas que los SRD utilizan comúnmente, las *Industrial, Scientific, and Medical* son bandas abiertas para uso general.

3. Análisis de requerimientos

A grandes rasgos, se desarrollará un sistema cuya componente principal sea un microcontrolador que se encargue de gestionar distintos módulos periféricos. Mismamente, los transductores que se permitan captar las señales IR y RF. Además, se contará con algún *display* y botonera para permitir al usuario seleccionar distintos modos de funcionamiento. Finalmente, alguna conexión con algún método de almacenamiento masivo como microSD. En la [Figura 1](#) se detalla este sistema como diagrama de bloques.

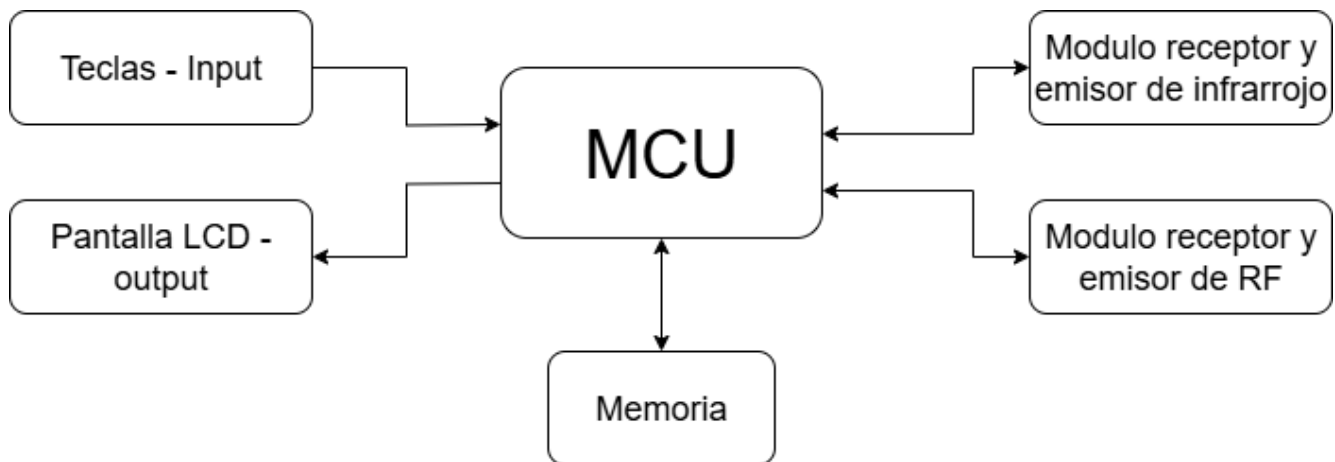


Figura 1. Diagrama de bloques de Médium.

A continuación, se detallan explícitamente los requerimientos del proyecto. A grandes rasgos, se dividieron según el ámbito al que corresponden: comunicación con el exterior (*periféricos*), control de los periféricos (*sistema*), y otros requerimientos sobre el proyecto (*proyecto*).

3.1. Requerimientos de los periféricos (funcionales)

- El sistema debe contar con un transductor de señales IR.
- El sistema debe ser capaz de demodular señales IR digitales.
- El sistema debe ser capaz de transmitir señales IR.
- El sistema debe contar con un transductor de señales de RF.
- El módulo RF debe poder captar señales en bandas ISM/SRD comunes.
- El módulo RF debe poder demodular señales digitales comunes (mínimamente OOK).
- El sistema debe ser capaz de transmitir señales RF.
- El dispositivo debe contar con un puerto para conectar algún dispositivo de almacenamiento masivo como microSD.
- El dispositivo debe contar con una pantalla para mostrar la interfaz de usuario.
- El dispositivo debe contar botones (o similares) para que el usuario interactúe con el mismo.

3.2. Requerimientos del sistema (funcionales)

- El sistema debe contar con una interfaz de usuario para seleccionar distintos modos de funcionamiento.
- La interfaz de usuario debe mostrarse en la pantalla.
- La interfaz de usuario debe responder a los botones presionados por el usuario.
- Cada periférico debe poder ser activado cuando el usuario lo necesite.
- El dispositivo debe permitir almacenar las señales captadas en el sistema de almacenamiento
- El dispositivo debe reproducir señales almacenadas a través de los periféricos.
- El sistema debe permitir eliminar señales almacenadas del sistema de almacenamiento.
- El sistema debe registrar la fecha y hora de las capturas realizadas para su posterior trazabilidad.

3.3. Requerimientos del proyecto (no funcionales)

- El proyecto debe desarrollarse utilizando el microcontrolador EDU-CIAA-NXP [2] como plataforma principal.

- El código debe estar desarrollado en C/C++, compatible con los entornos de trabajo de la EDU-CIAA-NXP
- Todos los elementos físicos deben estar ensamblados en un único dispositivo final.
- El diseño del dispositivo debe ser compacto y seguro para su manipulación.
- La interfaz diseñada para la pantalla debe ser clara y sencilla, priorizando la facilidad de uso.
- El transmisión y recepción de señales debe acatarse a las normativas del ENACOM vigentes a la fecha.
- El almacenamiento de señales debe realizarse según estándares preexistentes.
- La documentación del software debe incluir diagramas, descripciones de módulos y guías de uso para el usuario final.
- El proyecto debe estar finalizado, junto con su documentación correspondiente, antes del final de la cursada de “Taller de Proyecto I” del segundo semestre de 2025.
- El desarrollo del proyecto debe ajustarse a un presupuesto preestablecido.

4. Diseño de hardware

Se contará con distintos módulos de entrada/salida que se comunicarán con la EDU-CIAA-NXP, que se encargará de coordinarlos.

4.1. Diseño de Esquemático

4.1.1. Teclas (*entrada*)

Para la etapa de entradas de usuario se implementó un conjunto de dispositivos que permiten tanto el control digital como analógico del sistema.

En primer lugar, se utilizó un bloque analógico conformado por dos potenciómetros lineales que permiten registrar desplazamientos en coordenadas cartesianas. Cada potenciómetro se comporta como un divisor resistivo variable, de modo que al girar el eje se modifica la tensión presente en su terminal de salida. Esta variación de tensión se mide mediante ADC (*analog-to-digital converter*) integrados en la EDU-CIAA-NXP, utilizando los canales CH1 y CH2. De esta forma, es posible representar movimientos continuos en dos dimensiones. La finalidad de estos potenciómetros es permitir al usuario navegar por el menú y controlar la visualización de datos en pantalla.

En complemento, el módulo incluye un pulsador digital, lo que brinda una señal de entrada binaria (presionado / no presionado) que puede emplearse para acciones de control adicionales.

Además, se incorporaron dos pulsadores independientes, conectados a los pines digitales GPI02 y GPI04. En estos casos, se aplicará una rutina de anti-rebote por software y hardware, con el fin de garantizar la estabilidad de la lectura y evitar falsas detecciones ocasionadas por el rebote mecánico de los contactos.

Finalmente, el sistema dispone de un interruptor principal (*switch*) encargado de habilitar o inhabilitar la alimentación general. Este se conecta en serie entre el paquete de baterías y el regulador de tensión, actuando como interruptor maestro de encendido.

El conexionado del mismo se presenta en la [Figura 2](#).

4.1.2. Pantalla OLED (*salida*)

Para la etapa de salida de información se seleccionó un display OLED de 1,3 pulgadas con resolución de 128×64 píxeles. Este tipo de pantalla resulta adecuada para el proyecto, ya que permite visualizar datos numéricos, texto descriptivo y representaciones gráficas simples de las señales capturadas, todo en un formato compacto y de bajo consumo.

El principio de funcionamiento de la pantalla OLED se basa en diodos orgánicos emisores de luz, lo que brinda ventajas frente a pantallas LCD tradicionales: no requiere retroiluminación, posee alto contraste y ofrece un ángulo de visión amplio. La resolución de 128×64 es suficiente para mostrar información estructurada como menús, estados de conexión y tramas de señal.

La comunicación con la EDU-CIAA-NXP se realiza mediante el protocolo I2C, lo cual reduce el número de pines necesarios a solo dos líneas compartidas:

- I2C_SCL: línea de reloj generada por la EDU-CIAA-NXP.
- I2C_SDA: línea de datos bidireccional para transmitir la información.

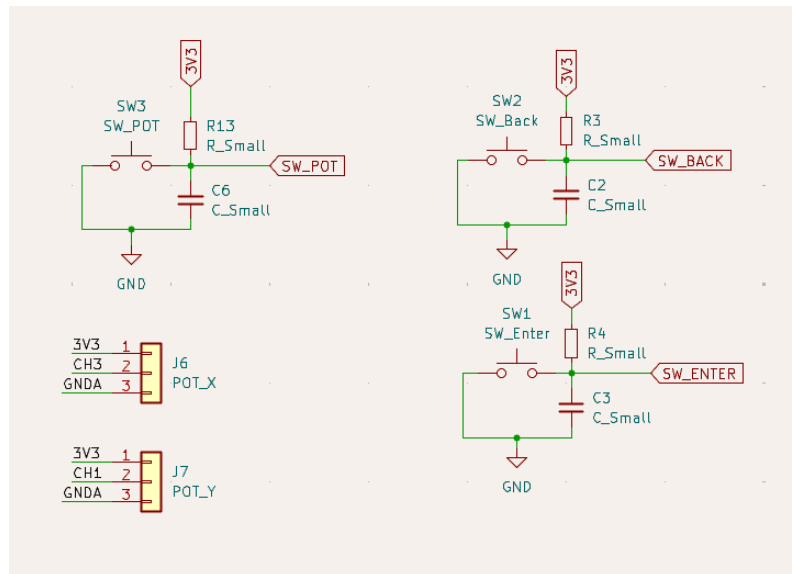


Figura 2. Esquemático de entradas.

El módulo se alimenta directamente con 3,3 V provistos por la propia placa, lo cual asegura compatibilidad eléctrica sin necesidad de conversores de nivel. Esta conexión de detalla en la Figura 3.

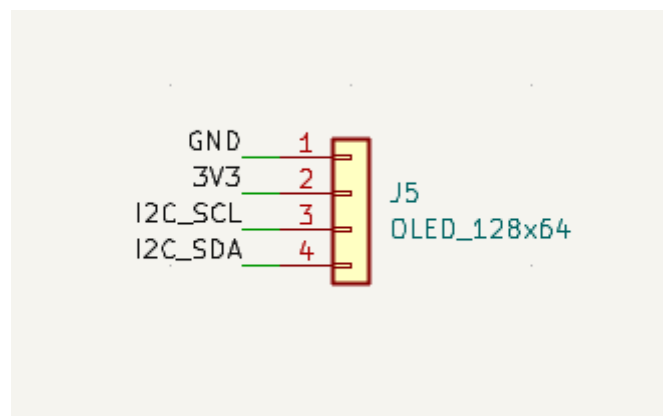


Figura 3. Esquemático del display OLED.

4.1.3. Módulo emisor/receptor infrarrojo (*entrada/salida*)

Para la captura de señales infrarrojas se eligió el módulo TSOP38438 como receptor IR. Este componente incorpora un filtro óptico y un demodulador interno sintonizado a 38 kHz, lo que permite obtener una señal digital ya procesada, más limpia y con mayor inmunidad al ruido eléctrico y a la luz ambiente. El módulo se alimenta con 5 V y entrega una salida digital activa en bajo cuando detecta una portadora infrarroja.

Para la transmisión se optó por un LED infrarrojo, acompañado por una etapa de amplificación basada en un transistor 2N2222 en configuración de conmutación. Esta etapa permite incrementar la corriente que atraviesa el LED, ajustando así la potencia y el alcance de la señal emitida. De esta manera, es posible calibrar la intensidad del transmisor para asegurar una correcta comunicación con distintos dispositivos. Este detalle se grafica en la Figura 4.

La interfaz eléctrica con la EDU-CIAA-NXP (diagramada en la Figura 5) se realizó de la siguiente manera:

- Receptor IR (TSOP38438) conectado a la entrada digital GPI07.
- Emisor IR (LED + 2N2222) controlado desde la salida digital GPI05.

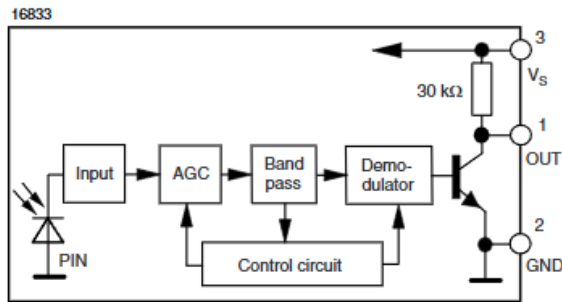


Figura 4. Esquemático interno de TSOP38438.

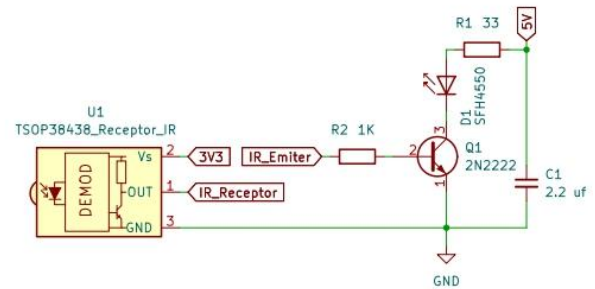


Figura 5. Esquemático módulo IR.

4.1.4. Módulo emisor/receptor de radiofrecuencia (*entrada/salida*)

Para la etapa de radiofrecuencia se utiliza el transceptor CC1101, alimentado a 3,3 V y conectado por SPI. Dentro del proyecto, el módulo se opera en la banda 433,92 MHz, con captura/replay OOK asíncrono.

En firmware se definieron dos perfiles de modulación soportados para el CC1101:

- AM650
- AM270

Estos perfiles se seleccionan desde la interfaz al momento de grabar en RF y se guardan como metadata dentro del archivo .sig, para que en la reproducción se pueda restablecer automáticamente la misma configuración de modulación.

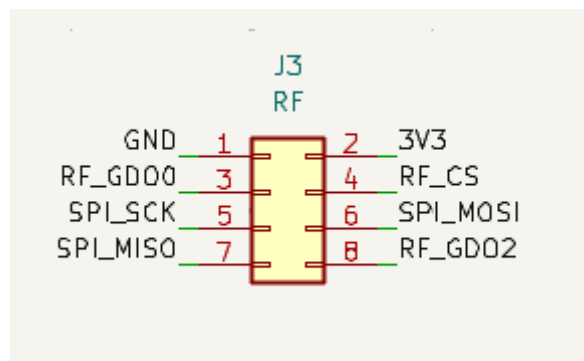


Figura 6. Esquemático módulo Radiofrecuencia.

La elección del CC1101 se justifica por:

- La flexibilidad en protocolos: permite captar tanto tramas estándar como propietarias.
- El bajo consumo de energía, adecuado para sistemas embebidos.

El módulo se conecta a la EDU-CIAA-NXP a través de la interfaz SPI0, compartida también con el módulo microSD. En este esquema, cada dispositivo cuenta con un pin de *chip select* independiente, permitiendo que la EDU-CIAA-NXP seleccione cuál de los periféricos SPI está activo en cada momento. Y además este módulo posee dos pines RF_GDO0 y RF_GDO2 que se conectan a pines GPIO8 y GPIO6 respectivamente, que sirven para recibir interrupciones del CC1101 (ej: “paquete recibido” o “buffer vacío”).

El CC1101 trabaja con señales de radiofrecuencia analógicas en bandas ISM/SRD, pero a nivel de microcontrolador se controla digitalmente por SPI y por sus pines GDO. De esta manera, la EDU-CIAA-NXP configura frecuencia/modulación, captura los tiempos de nivel lógico y luego puede reconstruir la señal para replay.

4.1.5. Memoria microSD (*entrada/salida*)

El módulo microSD es utilizado en este proyecto como memoria externa de almacenamiento persistente. Su función principal es guardar las señales captadas (tanto IR como RF) y los patrones decodificados para su posterior análisis, visualización o reproducción.

Se usó un modelo genérico sin nombre coloquialmente conocido como “microSD card module”, popularizado por su compatibilidad con Arduino. Cuenta con una interfaz de comunicación SPI y su tensión de alimentación 5 V. Su consumo típico se encuentra entre 20 mA y 50 mA según la operación de lectura/escritura.

La conexión del módulo microSD a la EDU-CIAA-NXP se realiza a través de la interfaz SPI0, compartida también con el módulo CC1101. La comunicación está soportada por la biblioteca *FatFs* incluida en el `firmware_v3` [3], lo que asegura compatibilidad con sistemas de archivos FAT16/FAT32.

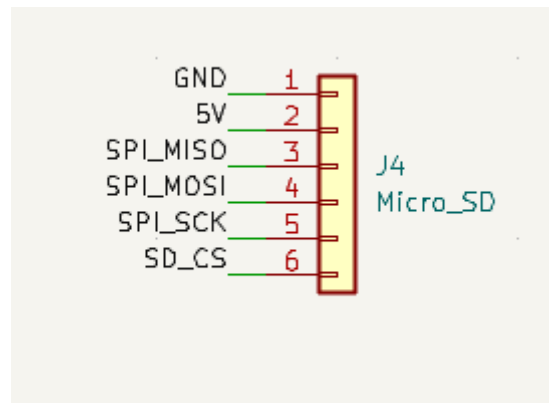


Figura 7. Esquemático módulo microSD.

4.1.5.1. Modificaciones

Al conectar el módulo de microSD y el CC1101, se detectaron problemas en el bus SPI. Luego de depurar, se concluyó que el problema provenía del módulo de microSD: el mismo no libera la línea MISO correctamente. Según se logró investigar, usuarios de foros de ayuda atribuyen el problema al *level shifter* incluido.

Este componente se agrega para elevar los 3,3 V de salida nativos de la microSD a 5 V para ser compatible con el GPIO de Arduino. Estos módulos además cuentan con una tecla para alternar entre las distintas tensiones de salida.

Como el GPIO de la EDU-CIAA-NXP trabaja a 3,3 V, no es necesario utilizar el *level shifter*. Así, se realizó un puente que lo evita, como se puede apreciar en la Figura 8.



Figura 8. Modificación del módulo de microSD.

4.1.6. Alimentación

El sistema se alimentará mediante un banco de pilas para mantener la portabilidad del prototipo. El banco provee un voltaje nominal cercano a 10 V, por lo que se adopta la siguiente arquitectura de alimentación:

- Entrada del pack de pilas → Bornera de entrada.
- Regulador DC-DC 10 V → 5 V (principal). El regulador entrega la alimentación a la EDU-CIAA-NXP y a los módulos que aceptan 5 V.
- Regulador adicional 5 V → 3,3 V. Este regulador se incorpora para alimentar de forma dedicada los periféricos que operan a 3,3 V. De esta manera se evita sobrecargar la salida de 3,3 V de la EDU-

CIAA-NXP, garantizando estabilidad en la alimentación y protegiendo tanto a la placa como a los dispositivos externos.

En la [Tabla 1](#) se detallan consumos estimados por bloque, el cálculo total y las decisiones de diseño.

Tabla 1. Consumo de los componentes.
Otros refiere a resistencias divisoras nivel, anti rebote, y demás componentes pasivos.

Componente	Corriente típica [mA]	Notas
EDU-CIAA-NXP	150	Consumo típico en operación.
CC1101	30	Durante transmisión.
microSD	100	Picos hasta 100 mA en escritura.
OLED 1.3" 128×64	30	Brillo medio, picos mayores si contraste al máximo.
TSOP38438	5	Módulo receptor (pequeño).
IR LED	100	Corriente pico cuando transmite.
Potenciómetros y pulsadores	5	Consumo pasivo pequeño.
Otros	10	Pérdidas menores.
Total nominal	440	

4.1.7. Resumen

El esquemático completo puede consultarse en el [Apéndice A.3](#). Además, en el [Apéndice A.1](#) se presenta la lista completa de insumos del proyecto.

4.2. Diseño del circuito impreso

El circuito impreso desarrollado presenta las características técnicas de la [Tabla 2](#). El diseño fue pensado para acoplarse mecánica y eléctricamente a la placa EDU-CIAA-NXP, utilizando conectores dobles de 2x20 pines para la interconexión.

Tabla 2. Características del PCB.

Propiedad	Valores
Ancho	120 mm
Alto	85 mm
Cantidad de capas	1
Tipo de capas	simple faz
Ancho de pista	1 mm
Ø interno de los pads	0,7 mm
Ø externo de los pads	1,7 mm

4.2.1. Proceso de diseño del PCB

El proceso de diseño comenzó con la definición de las restricciones físicas, estableciendo las dimensiones totales de la placa y la ubicación de los conectores de interconexión con la EDU-CIAA. Una vez fijadas estas condiciones, se procedió al posicionamiento de los componentes, agrupando los circuitos según su función y procurando que aquellos que interactúan entre sí se mantuvieran próximos en el PCB. Este criterio redujo la longitud de las pistas y facilitó el enrutado, además de disminuir el riesgo de interferencia entre señales.

Posteriormente, se inició el ruteo de las conexiones, comenzando por las pistas más simples y de menor recorrido, para luego abordar las conexiones más extensas o críticas. En esta etapa se realizaron ajustes en la asignación de pines hacia la EDU-CIAA, buscando minimizar la cantidad de cruces y puentes

necesarios. Se dio prioridad a las líneas de señal frente a las de alimentación, a fin de garantizar una correcta transmisión de datos y una disposición estable del sistema.

Debido a la limitación de trabajar en una sola cara, fue necesario incorporar algunos puentes de conexión, principalmente en el bus SPI, en la alimentación de 3,3 V del módulo RF y para unir dos islas de GND que quedaban aisladas. Estas soluciones permitieron mantener la integridad del diseño sin comprometer la funcionalidad ni la estética del circuito.

El resultado final es una placa compacta, ordenada y funcional, que integra de manera eficiente todos los módulos del sistema: el módulo RF, el receptor y emisor IR, el módulo microSD, la pantalla OLED, los potenciómetros de entrada, los pulsadores, y el sistema de regulación de tensión. En el [Apéndice A.4](#) se detallan los diseños PCM.

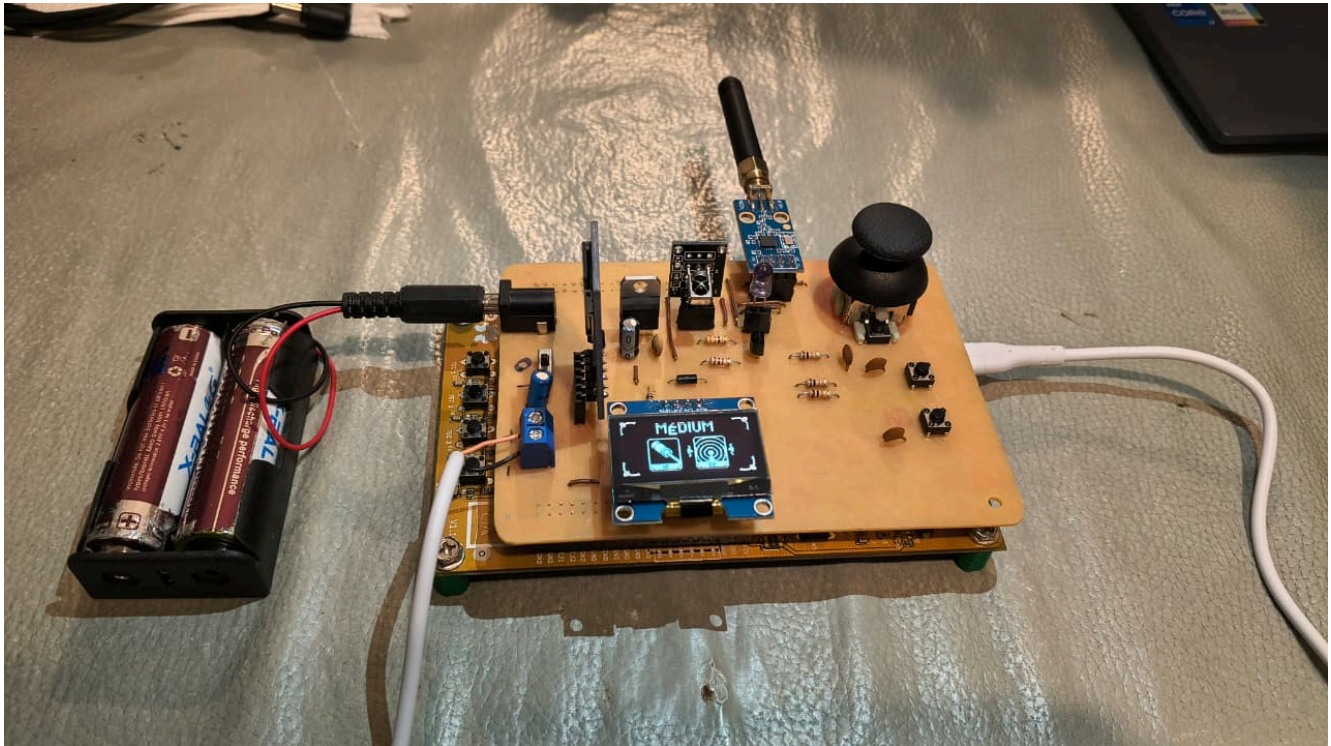


Figura 9. Fotografía de *Médium* con todos sus componentes.

5. Diseño del firmware

Se desarrolló un firmware trabajar con todos los módulos presentados. El mismo se encuentra publicado en <https://github.com/santi-nihany/medium>.

5.1. Arquitectura general

El sistema corre sobre FreeRTOS en modo *preemptive*, con una arquitectura simple basada en dos tareas principales: **Storage** y **UI**. La captura y reproducción IR/RF se ejecuta desde la lógica de la tarea de interfaz, mientras que la tarea de almacenamiento se encarga del monitoreo periódico de la microSD. Para evitar conflictos en periféricos compartidos, el acceso al bus SPI (microSD + CC1101) se arbitra con un mutex global estático. Desde la interfaz, el usuario puede grabar, reproducir y borrar señales en slots fijos.

5.1.1. Diagrama de flujo

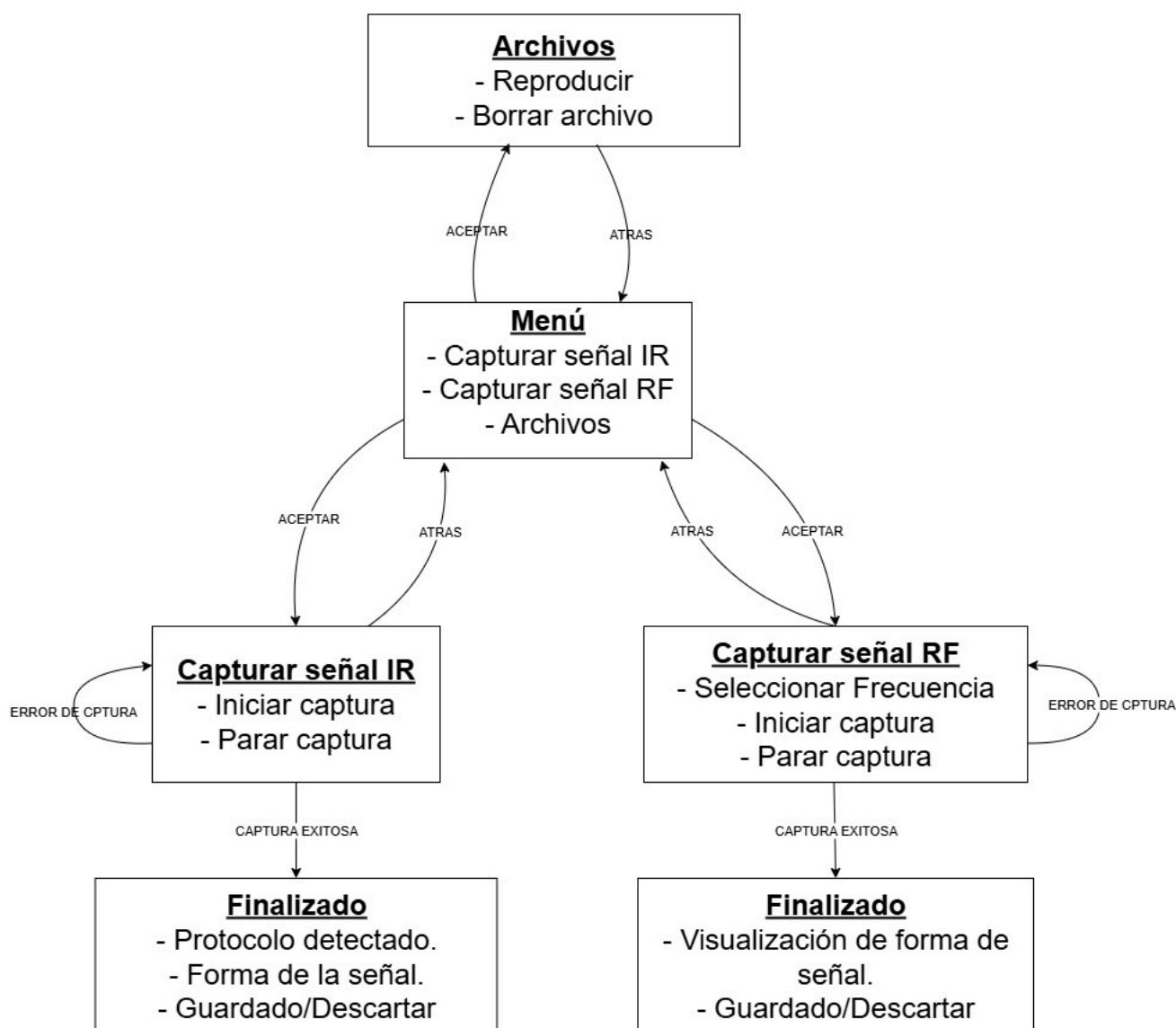


Figura 10. Diagrama de flujo de la interfaz de usuario.

Más detalle sobre cómo el usuario interactúa con el sistema en el [Apéndice A.5](#).

1. Programa principal
 - Configura el hardware (timers, GPIOs, SD, UART).
 - Inicializa módulos (``buttons``, ``display``, ``storage``, ``ir``, ``rf``) y el mutex SPI.
 - Crea dos tareas: ``Storage`` y ``UI``.
 - Arranca el scheduler → el RTOS empieza a planificar tareas.
2. Task ``Storage`` (prioridad media-alta)
 - Ejecuta periódicamente ``storageUpdate()``.
 - Detecta cambios de estado de la microSD (insertada/no disponible).
 - Realiza un ``probe`` periódico del filesystem para detectar extracción sin pin CD.
3. Task ``UI`` (prioridad media)
 - Lee joystick y botones (ENTER/BACK).
 - Gestiona el menú en LCD.
 - Para grabación: ejecuta captura IR o RF, serializa a formato ``.sig`` y guarda en microSD.
 - Para reproducción: carga ``.sig`` desde microSD y llama a ``irReplayEdges()`` o ``rfReplayEdges()``.
 - Para borrado: elimina el archivo asociado al slot.
4. Módulos IR/RF
 - Implementan captura y replay con cancelación desde callback de UI.
 - En RF se permite seleccionar frecuencia/modulación antes de grabar y se persiste metadata en el ``.sig``.
5. Almacenamiento
 - Se usa FatFS para lectura/escritura de archivos ``.sig``.
 - El acceso SPI está protegido por mutex para coexistencia SD/CC1101.
 - Los slots son fijos por modo: ``IR1.sig`` a ``IR5.sig`` y ``RF1.sig`` a ``RF5.sig``.

Código 1. Pseudocódigo del diagrama de la [Figura 10](#).

5.1.2. Bibliotecas externas y dependencias

Bibliotecas del proyecto CIAA

- sAPI v0.6.2

Librerías de terceros

- FreeRTOS v10 — kernel RTOS preemptivo.
- FatFS — sistema de archivos para microSD (`f_write`, `f_open`, `f_close`).

5.2. Módulos desarrollados

5.2.1. Sistema RTOS

Se implementó un esquema RTOS compacto con dos tareas de aplicación: **Storage** y **UI**. A diferencia de versiones anteriores, en la versión actual no se utiliza una arquitectura por colas/stream buffers entre tareas para IR/RF; las operaciones de captura, serialización y replay se ejecutan dentro de la tarea de UI, y la tarea de almacenamiento queda dedicada al estado de la microSD.

5.2.1.1. Primitivas de RTOS creadas

Se utilizan las siguientes primitivas de FreeRTOS:

- **Tasks:**
 - **Storage** (prioridad `tskIDLE_PRIORITY + 3`), período base de 20 ms, con *probe* de filesystem cada 100 ms.
 - **UI** (prioridad `tskIDLE_PRIORITY + 2`), lazo principal de interacción cada 40 ms.
- **Mutex estático del bus SPI:** arbitra el acceso concurrente entre microSD y transceptor CC1101.
- **Delays y ticks de scheduler:** usados para temporización de interfaz, debounce y ventanas mínimas de visualización.

5.2.1.2. Pruebas

Como primera verificación, se comprobó por UART el arranque correcto del *scheduler* y la creación de las tareas **Storage** y **UI**. También se validó el comportamiento esperado por prioridad: la tarea de

almacenamiento mantiene el monitoreo periódico de SD, y la UI conserva la respuesta del menú con ciclos de 40 ms sin bloquear el sistema.

Como segunda prueba, se validó el flujo completo de usuario en ambos modos (IR y RF): captura con posibilidad de cancelación, guardado en archivo `.sig`, reproducción desde slot y borrado. En RF, además, se verificó la selección previa de frecuencia/modulación y su persistencia como metadata TLV dentro del archivo.

Se verificó por UART que:

- 1) El estado de la microSD cambia correctamente entre “lista” y “no disponible”.
- 2) Las operaciones de guardado/carga de `.sig` finalizan correctamente para IR y RF.
- 3) El sistema detecta cancelación de captura por botón o por retiro de SD sin quedar bloqueado.
- 4) La reproducción respeta el tipo de señal del archivo y aplica metadata RF cuando está disponible.

5.2.2. Pantalla OLED

Para la pantalla OLED con *driver* SH1106 se desarrollaron dos librerías. La primera, `sh1106.h` se encarga de la comunicación con la pantalla en sí utilizando I²C y encapsula todas las particularidades del dispositivo. La segunda, `display.h` es una librería de más alto nivel que consume `sh1106.h` y expone funciones para dibujar líneas, rectángulos, pegar imágenes y demás.

Para minimizar el uso de memoria utilizada, se guardan distintos *sprites* en la memoria de programa. Además, también se cuenta con la capacidad de escribir texto arbitrario con una tipografía predefinida.

En la [Figura 11](#) se muestra la pantalla mostrando el menú de inicio del programa.

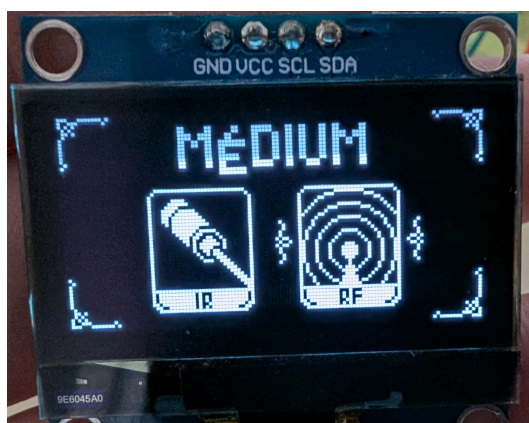


Figura 11. Pantalla OLED mostrando el menú de inicio.

5.2.3. MicroSD

Se integró FatFS con el sistema RTOS descrito en la sección anterior. En la versión actual, los archivos se gestionan por slots fijos (`IR1.sig ... IR5.sig` y `RF1.sig ... RF5.sig`). La tarea UI solicita guardado/carga/borrado según la acción elegida por el usuario, y el módulo `storage` ejecuta I/O sobre FatFS con exclusión mutua del bus SPI para convivir con el CC1101. Se verificó escritura y lectura exitosa en ambos modos, incluyendo metadata de RF (frecuencia y modulación) dentro del formato `.sig`.

Además, para serializar las capturas se definió un formato binario propio `.sig`, implementado en `program/src/utils/sig.c`. Como herramienta de desarrollo y depuración se agregó también el script `scripts/custom/signals.py`, que permite validar e inspeccionar los archivos generados por firmware desde PC. En el [Apéndice A.6](#) se detalla la estructura del formato.

En la [Figura 12](#) se observa la salida por UART durante la ejecución del sistema, donde se evidencia el monitoreo de SD y las operaciones de guardado/carga en la microSD.

```
hello from signal replay
hello from ui controller
[MockGen] Generating burst #4 (64 samples)...
[MockGen] Burst #4 sent to StreamBuffer
[CaptureIR] Burst complete: 64 samples captured
[CaptureIR] Packet (64 samples) sent to queue
[Storage] === PACKET #2 ===
[Storage] Mode: IR, Samples: 64
[Storage] Writing to: SDC:/signals/signal_IR_000001.sig
[Storage] SUCCESS: Wrote 293 bytes
[Storage] Files saved: 2/5
[Storage] Heap: 2680 bytes
[Storage] =====
[Housekeeping] Free heap: 2680 bytes, Min: 2680 bytes
hello from signal replay
[Housekeeping] Free heap: 2680 bytes, Min: 2680 bytes
hello from ui controller
[MockGen] Generating burst #5 (64 samples)...
[MockGen] Burst #5 sent to StreamBuffer
[CaptureIR] Burst complete: 64 samples captured
[CaptureIR] Packet (64 samples) sent to queue
[Storage] === PACKET #3 ===
[Storage] Mode: IR, Samples: 64
[Storage] Writing to: SDC:/signals/signal_IR_000002.sig
[Storage] SUCCESS: Wrote 293 bytes
[Storage] Files saved: 3/5
[Storage] Heap: 2680 bytes
[Storage] =====
[Housekeeping] Free heap: 2680 bytes, Min: 2680 bytes
```

Figura 12. Logs de monitoreo RTOS y operaciones de almacenamiento en microSD.

5.2.4. Modulo IR

En los sistemas de comunicación por infrarrojo, la información digital no se transmite sobre una señal infrarroja continua, sino que como una señal modulada sobre una portadora de alta frecuencia. En la gran mayoría de controles comerciales esta portadora es aproximadamente de 38 kHz, por eso se decidió hacer que el modulo detecte/emita señales en esta frecuencia.

La utilización de una portadora presenta varias ventajas:

- Permite a los receptores IR comerciales (por ejemplo, módulos tipo TSOP, KY-022, VS1838B) filtrar la luz ambiente y el ruido proveniente de fuentes como lámparas o el sol.
- Simplifica la detección de la señal útil, ya que el receptor entrega una salida digital ya demodulada.
- Aumenta la robustez del sistema frente a interferencias.

Desde el punto de vista lógico, **el receptor no entrega la portadora, sino una señal digital** que indica presencia o ausencia de trenes de 38 kHz.

La información binaria (bits 0 y 1) se codifica mediante la duración de los pulsos infrarrojos, utilizando combinaciones de:

- MARK: período en el que la portadora de 38 kHz está presente (LED IR encendido).
- SPACE: período en el que no hay emisión infrarroja (LED apagado).

La diferencia entre un 0 y un 1 no se da por el nivel lógico, sino por la duración del espacio posterior al pulso activo.

Existen múltiples protocolos de transmisión infrarroja, desarrollados por distintos fabricantes y estandarizados de hecho en la industria. Algunos de los más conocidos son:

- NEC
- RC-5 / RC-6 (Philips)
- Sony SIRC
- Panasonic

Cada protocolo define la frecuencia de la portadora, estructura temporal de la trama, la forma en la que se codifican los bits, el tamaño de mensaje etc. Esto implica que no existe una única forma universal de decodificar señales IR, sino que el software debe estar diseñado para un protocolo específico o ser lo suficientemente flexible para soportar varios.

5.2.4.1. NEC

El protocolo NEC es uno de los más utilizados y documentados, tanto en dispositivos comerciales como en proyectos educativos y académicos. Su popularidad se debe a su simplicidad, claridad temporal y bajo costo de implementación.

La estructura típica de la trama se detalla en la [Figura 13](#).

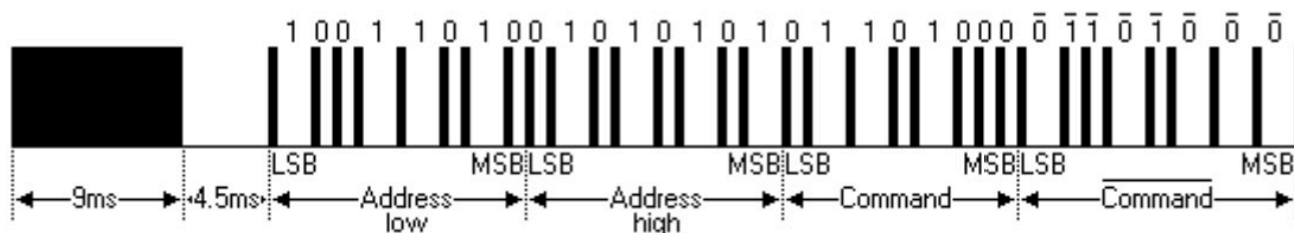


Figura 13. Trama NEC.

Se puede apreciar que la trama tiene un pulso de encabezado que da inicio (*header*) que consta de 9 ms de MARK y 4,5 ms de SPACE. Luego continua con 32 bits que se dividen en dos, 16 para mandar el address del emisor y 16 para el comando que se desea compartir. Para llevar un control de errores de los 16 bits, 8 son útiles para transportar datos y 8 son para control de errores ya que envía la misma señal pero invertida.

La codificación de bits es la de la [Tabla 3](#).

Tabla 3. Codificación de bits NAC (valores aproximados).

	0	1
MARK	560 μ s	560 μ s
SPACE	560 μ s	1690 μ s

5.2.4.2. Software

El diseño del software se centró en la implementación de un sistema capaz de recibir, decodificar y transmitir señales infrarrojas bajo el protocolo NEC, priorizando la robustez temporal y la facilidad de depuración, utilizando una estrategia de polling temporizado.

Esta decisión se tomó luego de evaluar el comportamiento del sistema bajo distintos enfoques y observar interferencias temporales al utilizar múltiples interrupciones simultáneas.

Arquitectura general del sistema

A nivel funcional, el software se divide en los siguientes bloques lógicos:

- 1) Adquisición de señal infrarroja (RX – Polling)
- 2) Procesamiento y decodificación del protocolo NEC
- 3) Generación de señal infrarroja (TX – Emisión NEC)
- 4) Gestión temporal mediante temporizadores
- 5) Programa principal (control de flujo y validación)

5.2.4.2.1. Adquisición de señal infrarroja (RX)

La recepción de la señal infrarroja se implementó mediante polling periódico del pin GPIO7, con un intervalo de muestreo fijo (SAMPLE_US). Esta técnica consiste en leer el estado lógico del pin a intervalos regulares y detectar transiciones de nivel comparando el estado actual con el anterior. Para medir la duración de cada pulso, se utiliza un temporizador de hardware configurado para contar microsegundos. Cada vez que se detecta un cambio de nivel en la entrada IR se hace lo siguiente:

- Se calcula el tiempo transcurrido desde el ultimo cambio.
- Se almacena el nivel lógico previo y su duración
- Se filtran pulsos demasiado cortos, considerados ruido o glitches.

Los pulsos válidos se almacenan en un buffer de estructuras `IRPulse_t`, que contiene dos campos: Nivel logico del pulso (MARK/SPACE) y la duracion en microsegundos.

El fin de trama se detecta cuando ya se han capturado almenos algunos pulso validos o el tiempo sin cambio en la entrada supera el TIMEOUT (definido como constante). Este criterio es consistente con el protocolo NEC, donde existe un silencio prolongado entre tramas.

5.2.4.2.2. Procesamiento y decodificación del protocolo NEC

La decodificación se realiza sobre el buffer de pulsos capturados, interpretando las duraciones conforme a la especificación del protocolo NEC. Es oportuno aclarar que el modulo KY-022 entrega la inversa de la señal emitida, es decir que constantemente esta en valor ALTO y cuando hay presencia de portadora su valor entregado pasa a estado BAJO.

Las etapas del algoritmo de decodificación son las siguientes:

- Validación de encabezado: Pulso bajo de aproximadamente 9ms y pulso alto de 4.5ms
- Decodificación de bits: cada bit se compone de un pulso bajo fijo de 560us y un pulso alto cuya duración determina si el bit es 0 o 1.
- Reconstrucción de la palabra de datos: Se reconstruyen 32 bits enviados LSB primero y se separan los campos.
- Verificación de integridad: Se comprueba que cada byte coincida con el complemento de su inverso. Si la verificación falla, la trama se descarta.

Este enfoque permite asegurar que únicamente se acepten tramas NEC válidas, descartando señales corruptas o protocolos incompatibles.

5.2.4.2.3. Generación de señal infrarroja (TX – Emisión NEC)

La transmisión de señales infrarrojas se implementó en dos niveles, como base la generación de la portadora por software alternando el estado del pin de salida a intervalos equivalentes a medio período de la frecuencia deseada ($\approx 13\mu s$ para 38 kHz). Y por complemento la emisión de la trama siguiendo estrictamente la estructura del protocolo NEC.

Para garantizar la precisión temporal durante la transmisión, se utiliza un enfoque bloqueante, deshabilitando interrupciones durante la emisión completa de la trama. Esto asegura que:

- No haya jitter introducido por otras tareas.
- Las duraciones de MARK y SPACE se mantengan dentro de los márgenes del protocolo.

5.2.4.2.4. Generación temporal

El temporizador **TIMER2** se configura como un contador libre en microsegundos y se utiliza como base de tiempo global del sistema. Su función principal es:

- Medir la duración de pulsos infrarrojos recibidos.
- Detectar flancos de señal en el receptor IR.
- Determinar el fin de una trama por inactividad.
- Controlar retardos precisos durante la transmisión.

La función `delayInaccurateUs()` se utiliza exclusivamente durante la captura de señal por polling, actuando como un retardo de muestreo.

El **TIMER0** se utiliza para generar la portadora infrarroja de forma periódica, alternando el estado del pin de salida a intervalos equivalentes a medio período de la portadora. **TIMER0** se configura para generar eventos que invierten el estado logico del pin **GPIO5** cada $13\mu s$.

El **TIMER3** se emplea para controlar la máquina de estados de transmisión NEC. Dispara callbacks al cumplirse la duración de cada segmento del protocolo. En cada callback se decide el próximo estado y la duración del siguiente intervalo.

5.2.4.2.5. Programa Principal

El programa principal se encarga de:

- Inicializar periféricos (**GPIO5/7**, **UART**, temporizador).
- Esperar la llegada de una señal infrarroja.
- Capturar y almacenar los pulsos recibidos.
- Decodificar la trama NEC.
- Mostrar los resultados por consola serie.
- Reemitir la trama decodificada como verificación funcional.

Este flujo permite validar el sistema de punta a punta, utilizando tanto un control remoto comercial como un receptor externo para confirmar la correcta transmisión.

6. Ensayos y mediciones

Para validar el correcto funcionamiento de *Médium* se realizaron distintos ensayos. Esta sección se centrará en los ensayos con los módulos más complejos (IR y RF).

6.1. Módulo de infrarrojos (IR)

6.1.1. Funcionamiento correcto de transmisión y recepción

En este escenario se validó el funcionamiento completo del sistema de comunicación infrarroja, verificando tanto la recepción como la reemisión de tramas NEC entre dos plataformas distintas.

Se configuro un Arduino como emisor de una trama infrarroja fija usando la librería IRremote.hpp. La trama emitida fue:

- Address: 0xA0
- Command: 0xE3

Una vez validada la trama recibida, la EDU-CIAA reemitió exactamente la misma trama infrarroja.

Finalmente, el Arduino decodificó la señal retransmitida por la EDU-CIAA y se verifico que sea la misma.

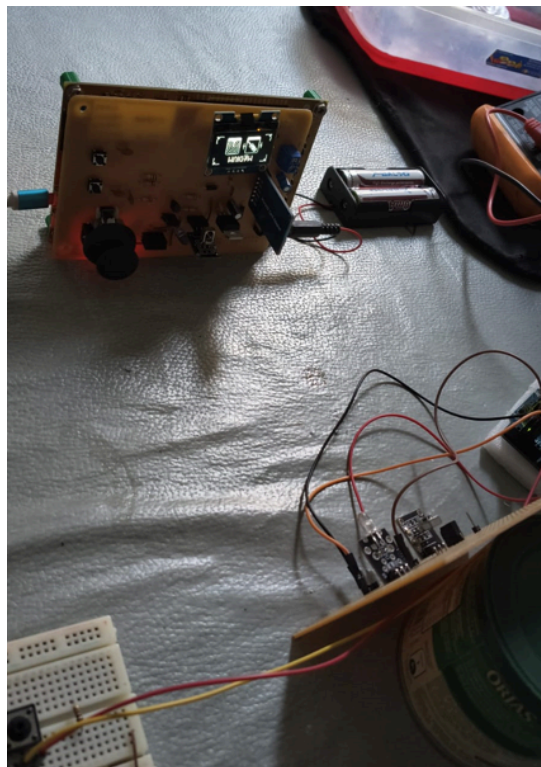


Figura 14. Escenario de prueba.


```
BOTON PRESIONADO -> Enviando NEC
TX Address: 0xA0 0b0000000010100000
TX Command: 0xE3 0b0000000011100011
```

Figura 15. Emisión desde Arduino.

Trama capturada (67 pulsos):		
[000]	Nivel=0	Dur=9053 us
[001]	Nivel=1	Dur=4402 us
[002]	Nivel=0	Dur=600 us
[003]	Nivel=1	Dur=551 us
[004]	Nivel=0	Dur=551 us
[005]	Nivel=1	Dur=551 us
[006]	Nivel=0	Dur=600 us
[007]	Nivel=1	Dur=601 us
[008]	Nivel=0	Dur=551 us
[009]	Nivel=1	Dur=601 us
[010]	Nivel=0	Dur=551 us
[011]	Nivel=1	Dur=600 us
[012]	Nivel=0	Dur=551 us
[013]	Nivel=1	Dur=1752 us
[014]	Nivel=0	Dur=501 us
[015]	Nivel=1	Dur=601 us
[016]	Nivel=0	Dur=550 us
[017]	Nivel=1	Dur=1752 us
[018]	Nivel=0	Dur=501 us
[019]	Nivel=1	Dur=1752 us
[020]	Nivel=0	Dur=551 us
[021]	Nivel=1	Dur=1702 us
[022]	Nivel=0	Dur=550 us
[023]	Nivel=1	Dur=1752 us
[024]	Nivel=0	Dur=501 us
[025]	Nivel=1	Dur=1752 us
[026]	Nivel=0	Dur=551 us
[027]	Nivel=1	Dur=1701 us

Figura 16. Recepcion en la EDU-CIAA.

```
[055] Nivel=1 Dur=1752 us
[056] Nivel=0 Dur=500 us
[057] Nivel=1 Dur=1752 us
[058] Nivel=0 Dur=501 us
[059] Nivel=1 Dur=1752 us
[060] Nivel=0 Dur=551 us
[061] Nivel=1 Dur=601 us
[062] Nivel=0 Dur=550 us
[063] Nivel=1 Dur=601 us
[064] Nivel=0 Dur=601 us
[065] Nivel=1 Dur=551 us
[066] Nivel=0 Dur=551 us
NEC decodificado OK! Addr=0xA0 Cmd=0xE3
NEC to send Addr=0xA0 Cmd=0xE3
```

Figura 17. Recepcion en la EDU-CIAA.

```
Protocol: NEC
Bits: 32
Decoded RAW HEX: 0x1CE35FA0
Decoded RAW BIN: 0b0001110011100011010111110100000
Raw length: 68
RAW buffer (mark/space en us):
MARK 8850
SPACE 4500
MARK 550
SPACE 550
MARK 500
SPACE 600
MARK 550
SPACE 600
MARK 500
SPACE 600
MARK 500
SPACE 600
MARK 500
SPACE 600
MARK 550
SPACE 1650
MARK 550
SPACE 550
MARK 550
SPACE 1700
MARK 550
SPACE 1700
MARK 500
SPACE 1700
```

Figura 18. Recepcion en Arduino.

Este ensayo confirma que:

- La implementación del decodificador NEC es correcta.
- La generación temporal de la trama NEC en la EDU-CIAA cumple con las tolerancias del protocolo.
- El sistema es interoperable entre distintas plataformas de hardware.

Este escenario valida el caso de uso principal del proyecto, demostrando un flujo completo.

6.1.2. Implementación alternativa con interrupciones y descarte de la solución

En una etapa inicial del desarrollo se implementó una versión del receptor infrarrojo basada en interrupciones por cambio de estado en el pin GPIO7, con el objetivo de detectar flancos de señal de forma inmediata. Descripción de la implementación

- Se configuró una interrupción por flanco en GPIO7.
- Cada cambio de nivel generaba una interrupción para medir la duración del pulso.
- El esquema funcionó correctamente en condiciones controladas, con solo el módulo receptor IR conectado a la EDU-CIAA.



Figura 19. Escenario interrupcion por flanco.

El problema surgió al integrar el sistema completo y conectar el shield con los demás módulos, se observó:

- Incremento significativo de ruido eléctrico en la línea de entrada.
- Generación de múltiples interrupciones espurias.
- Saturación del sistema de interrupciones.
- Bloqueo del programa principal debido a la alta frecuencia de eventos.

Este comportamiento provocó que el sistema dejara de responder correctamente, incluso ante señales válidas. Y como esta solución tenía que ser funcional en conjunto con los demás módulos, se decidió descartar la solución basada en interrupciones y adoptar un enfoque de polling.

6.1.3. Alimentación

Durante la etapa de ensayos del módulo infrarrojo se evaluó el comportamiento del sistema bajo distintas condiciones de alimentación eléctrica, con el objetivo de verificar su funcionamiento en escenarios reales de uso.

En una primera instancia, los ensayos se realizaron alimentando el sistema mediante un banco de pilas, cuya tensión medida fue de aproximadamente 6,8 V. Bajo estas condiciones, el módulo receptor infrarrojo KY-022 funcionó correctamente, permitiendo la detección de señales y la decodificación de tramas sin inconvenientes.

Sin embargo, al reemplazar el banco de pilas por un transformador de 9 V para alimentar el shield completo, se observó un comportamiento anómalo: el pin GPIO7 de la EDU-CIAA permanecía en un estado fijo y no detectaba ningún cambio de nivel lógico, aun cuando el módulo IR estaba recibiendo señales válidas.

Ante este escenario se intuyó que el problema venía del hardware y la alimentación. El módulo en arduino era funcional con 3,3 V y con 5 V. Entonces se consultó la bibliografía de la EDU-CIAA. A partir de este análisis se identificó que los pines GPIO de la EDU-CIAA operan en niveles lógicos de 0 V a 3,3 V y se determinó que el problema venía desde el diseño del esquemático.

Para resolver esta incompatibilidad de niveles de tensión, se realizó una modificación en el shield:

- Se interrumpió la pista original que conectaba el pin VCC del módulo KY-022 a la línea de 5 V, utilizando un corte físico (cutter).
- Se realizó un puente entre el pin VCC del módulo IR y la línea de 3,3 V, proveniente del regulador asociado al diodo Zener de la placa.

Luego de aplicar esta modificación:

- El sistema funcionó correctamente tanto con el banco de pilas como con el transformador de 9 V.

- El pin GPIO7 volvió a detectar correctamente los cambios de nivel.
- La decodificación y retransmisión de tramas infrarrojas se realizó sin errores.

6.2. Módulo de radiofrecuencia (RF)

6.2.1. Validación mínima de funcionamiento

Utilizando el módulo CC1101 y la librería `sapi.h` se realizó una comunicación básica con el transmisor FS1000A y el receptor MX-RM-5v. Se configuró el módulo RF para comunicarse en la frecuencia 433 MHz con modulación OOK.

Para la prueba de transmisión se utilizó la comunicación UART para elegir cuando transmitir un “1” o un “0” desde la terminal. En la [Figura 20](#) se observa la prueba del modulo RF como transmisor, en la foto se enciende la portadora y el receptor MX-RM-5v tiene un “1” en la salida, que se traduce en el led de salida encendido.

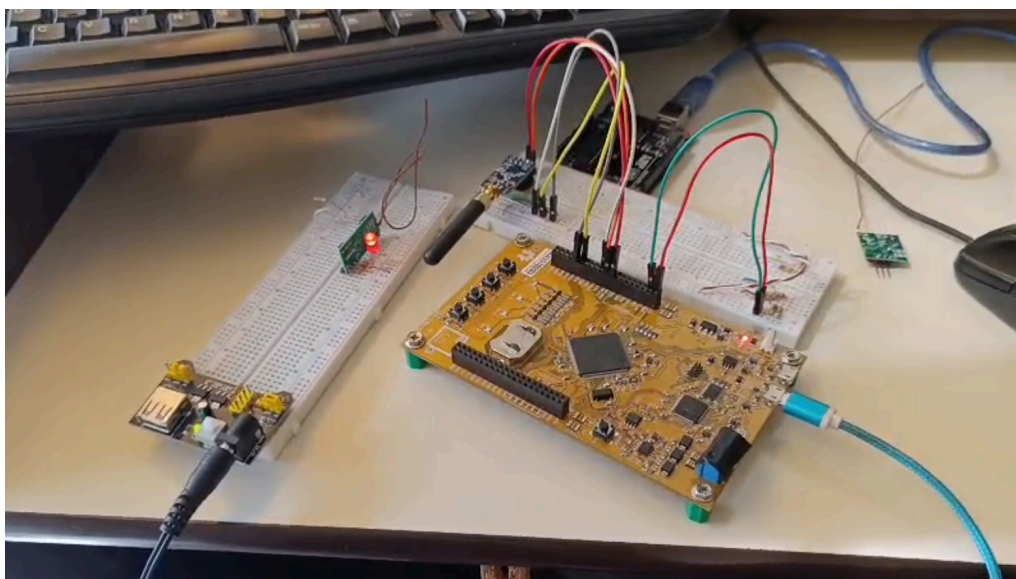


Figura 20. Prueba de modulo RF como transmisor.

Para la prueba de receptor se utilizó la comunicación UART para imprimir en la terminal continuamente el valor del pin GD02 del módulo RF, este estaba configurado para indicar de manera asincrónica el valor recibido. La prueba fue exitosa ya que al transmitir o no transmitir con el módulo FS1000A se representaba correctamente la salida en la terminal.

6.2.2. Primer ensayo: captura de señal de 315 MHz

Tabla 4. Configuración del módulo CC1101 para el primer ensayo.

Propiedad	Valor
Modo	recepción
Modulación	OOK
Frecuencia	315 MHz
Ancho de banda	812,5 kHz
Formato	serial asíncrono

Como objetivo se utilizó un timbre de RF *Candela 7279* el cual cuenta con un parlante con LED (receptor) y un botón que al presionarlo hace sonar el parlante (emisor). Este aparato se comunica en una frecuencia de 315 MHz, y es una comunicación sencilla de modulación OOK.



Figura 21. Timbre Candela 7279

Para realizar la captura, el módulo este replica la señal que capta por la antena en el pin **GD00**, por lo que se observó el valor del mismo repetidamente en intervalos regulares de tiempo para así obtener la forma de la señal que genera el emisor.

La forma de onda se presenta en la [Figura 22](#) (captura realizada con intervalo de 100 μ s), se intentó replicar para “duplicar el emisor” con el objetivo de activar el receptor pero no funcionó. Esto es debido a que no se capturó correctamente la señal transmitida por el emisor, se puede observar en la forma de onda que no tiene ningún patrón aparente.

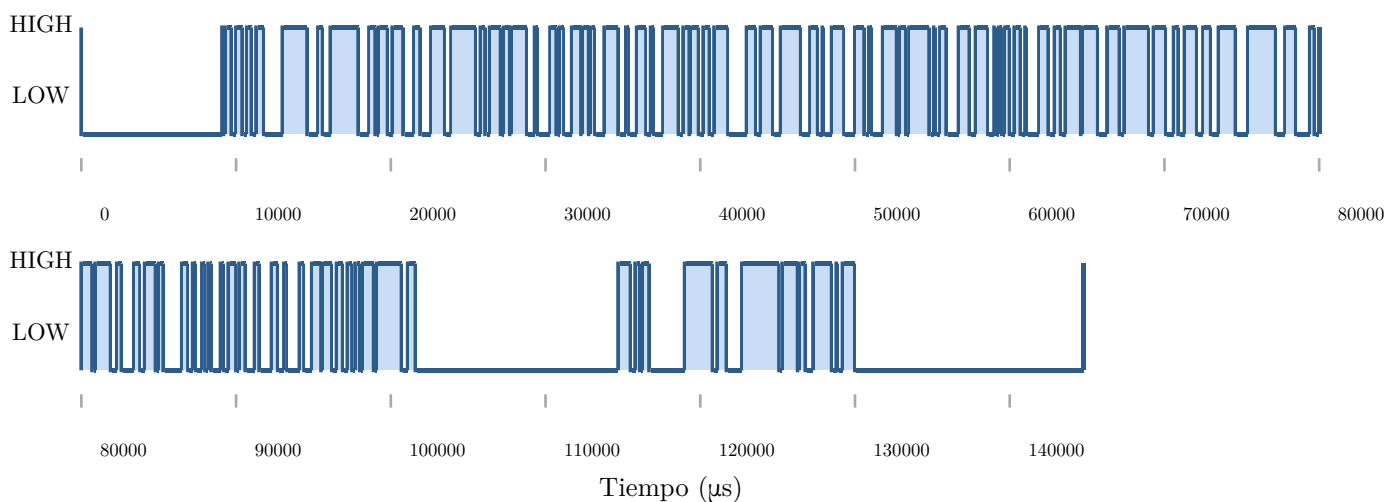


Figura 22. Forma de onda generada por el timbre Candela 7279.

6.2.3. Segundo ensayo: captura de señal de 433,92 MHz

Tabla 5. Configuración del módulo CC1101 para el segundo ensayo.

Propiedad	Valor
Modo	recepción
Modulación	OOK
Frecuencia	433,92 MHz
Ancho de banda	101 kHz
Formato	serial asíncrono

Como objetivo se utiliza un control remoto de portón eléctrico marca ZAP. Este cuenta en su interior con el chip **Si4010-C2** que cuenta con las siguientes características:

Tabla 6. Características del chip SI4010-C2.

Característica	Valor
Rango de frecuencia	27 MHz - 960 MHz
Modulación	FSK/OOK
Tasa de símbolos	Hasta 100 kbps
Codificación	Código fijo y código rolante

Este chip es aplicado en sistemas de:

- Controles de puertas y portones
- Llaves remotas sin cerradura
- Domotización y seguridad del hogar
- Controles remotos inalámbricos

En este caso es utilizado para un portón eléctrico en Argentina. La configuración mas común para estos casos es utilizar una frecuencia de 433,92 MHz, modulación OOK y codificación de código fijo.



Figura 23. Control remoto de portón eléctrico ZAP.

El método de codificación es lo mas importante en cuanto a seguridad. Si se utiliza código rolante cada vez que el control remoto se comunice con el portón se enviará un código distinto perteneciente a una lista de códigos que conocen ambos dispositivos, como cada vez se utiliza un código nuevo este no puede ser replicado.

Si se utiliza código fijo el emisor siempre transmite la misma señal, con el mismo código y secuencia de bits. Si la señal es capturada puede ser replicada y enviada al receptor en cualquier momento para que este realice la acción que tenga programada, la cual puede ser abrir un portón.

El fin de este ensayo es capturar y analizar la señal emitida por el control remoto.

Dado que no se conocía la frecuencia de operación del control remoto, se realizaron capturas iniciales comenzando en 433 MHz, frecuencia comúnmente utilizada en este tipo de dispositivos, con un ancho de banda de 812,5 kHz. Sin embargo, no se obtuvo información significativa.

Posteriormente, se modificó la configuración del módulo CC1101 para realizar capturas en 433,92 MHz, donde se logró detectar una señal, aunque con interferencias por ruido del entorno. En consecuencia, se redujo progresivamente el ancho de banda hasta alcanzar los 101 kHz, punto en el cual se obtuvo una señal clara.

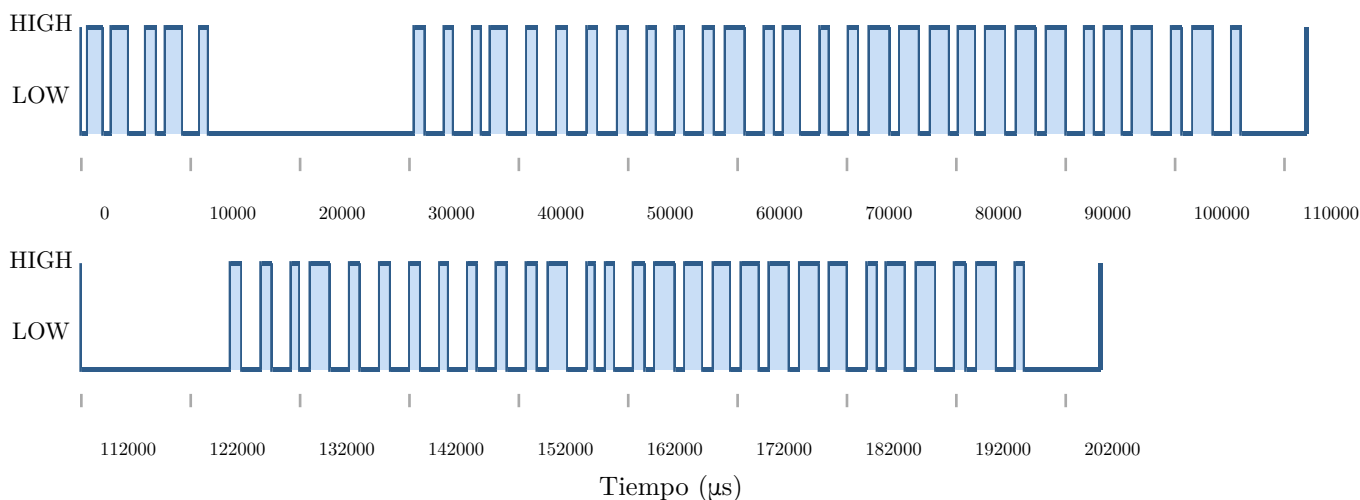


Figura 24. Forma de onda generada por el control remoto de portón ZAP.

La señal capturada se presentada en la figura [Figura 24](#), en la cual se puede observar 2 bloques de datos idénticos transmitidos por el control remoto.

Esto indica que el control remoto utiliza *código fijo*, la misma señal que se transmite en el bloque de tiempo 30.000 μ s - 110.000 μ s se observa en el bloque de tiempo 120.000 μ s - 200.000 μ s.

6.2.4. Tercer ensayo: ataque de interferencia (*jamming*)

Tabla 7. Configuración del módulo CC1101 para el tercer ensayo.

Propiedad	Valor
Modo	transmisión
Modulación	OOK
Frecuencia	315 MHz
Ancho de banda	812,5 kHz
Formato	serial asíncrono

El objetivo de este ensayo es interferir en la comunicación del timbre *Candela 7279* para que al presionar el botón no se active el parlante.

El módulo CC1101, configurado en modo de transmisión serial asincrónica, replica en la portadora el estado lógico presente en el pin **GD00**. Es decir, cuando **GD00** se encuentra en nivel **BAJO** el módulo desactiva la portadora, en cambio, si se encuentra en nivel **ALTO** la portadora permanece activa.

Para este ensayo, el pin **GD00** se mantiene en nivel **ALTO** de forma constante, forzando la transmisión continua en la misma frecuencia utilizada por el timbre, con el objetivo de generar interferencia y obstaculizar su comunicación enmascarando la transmisión del emisor.

El timbre *Candela 7279* posee un alcance de hasta 100 metros. Para evaluar el ataque de interferencia, se ubicó el módulo CC1101 junto al receptor del timbre.

Partiendo de una distancia de centímetros entre el emisor y el receptor, se presionaba el botón del control remoto mientras se aumentaba progresivamente la distancia hasta que el timbre dejaba de responder. En el momento en que el receptor dejaba de funcionar, se alejaba el CC1101 del mismo, observando que el timbre volvía a operar con normalidad, lo que indicaba que la señal generada por el módulo dejaba de enmascarar la transmisión del emisor.

La distancia máxima a la cual el ataque resultó efectivo mientras el módulo estaba junto al receptor fue de aproximadamente 6 metros. De esta manera, se confirmó el correcto funcionamiento del ataque de interferencia.

6.2.5. Notas adicionales de los ensayos

Durante las pruebas con el timbre se identificaron dos inconvenientes:

- 1) Al intentar capturar la señal transmitida por el timbre *Candela 7279*, no se detectó ningún patrón reconocible, la señal observada presentaba características similares a ruido.
- 2) Al ejecutar el ataque de interferencia, fue necesario acercarse considerablemente la antena del módulo *CC1101* al receptor del timbre *Candela 7279*, ya que a mayor distancia la interferencia no producía efecto.

Se considera que estos problemas podrían deberse al uso de una antena con una longitud no adecuada para la frecuencia de operación.

El módulo *CC1101* utilizado cuenta con una antena de 17,3 cm, correspondiente a la longitud óptima para una frecuencia de 433,92 MHz. Sin embargo, para una frecuencia de 315 MHz, la longitud de antena recomendada es de aproximadamente 23,8 cm.

Para poner a prueba esta hipótesis, se deberá reemplazar la antena del módulo *CC1101* por una de longitud adecuada y repetir los mismos ensayos, con el objetivo de evaluar si se obtienen resultados diferentes.

6.3. Ensayos de integración

Para verificar el funcionamiento integral del dispositivo, se realizaron varios ensayos con distintos dispositivos cotidianos. Principalmente, se buscó poder interceptar y replicar sus señales.

Para IR, se confeccionó una tira LED con un Arduino, cuatro LED y un control remoto, como se aprecia en la [Figura 25](#). El mismo cuenta con varios comandos que realizan distintas acciones.

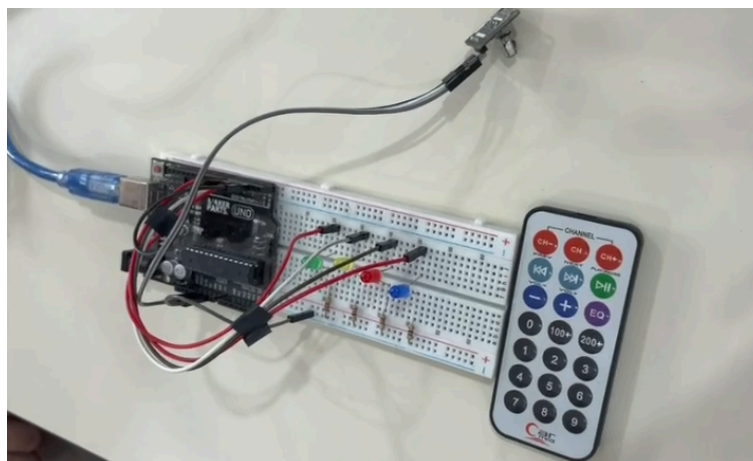


Figura 25. Emulador de tira de LED con control.

Con *Médium* se logró replicar correctamente el botón “1”. El análisis del mismo puede apreciarse en la [Figura 26](#), realizado con *signals.py*. Se puede apreciar en la captura los metadatos de NEC, incrustados por *Médium* sobre el archivo final. Además, *signals.py* grafica “0” y “1” cuando detecta que la señal es NEC para ayudar a visualizar el paquete. Particularmente, se verifica que se envían ocho “0” seguidos de ocho “1” (eso es, envía el *address* seguido del *address* negado) y *0x0c* seguido de *0xf3* (eso es, envía el *command* seguido del *command* negado).

Luego, se probó a captar el control remoto ZAP (de la [Figura 23](#)). El mismo se captó a 433,92 MHz con una modulación AM650 (OOK con un ancho de banda de 650 kHz) y se graficó la [Figura 27](#) con *signals.py*. Sobre esta señal no se puede apreciar nada en específico porque no se logró descubrir qué protocolo utiliza. sí se puede apreciar dos tipos de pulsos equiespaciados: uno corto y uno largo. Eso probablemente resulte en “0” y “1” digitales — seguramente no todos sean la clave en sí, ya que el protocolo probablemente incluya algunos bits de verificación.

Estas pruebas verifican el correcto funcionamiento de *Médium* para estos dispositivos, pero no para todos. En el caso de RF, no se logró replicar la señal de un timbre inalámbrico que alega trabajar a 433 MHz — se intentó captar en OOK y en FSK sin resultados satisfactorios. Si se quisiera ampliar la gama de dispositivos compatibles, se deben realizar más pruebas.

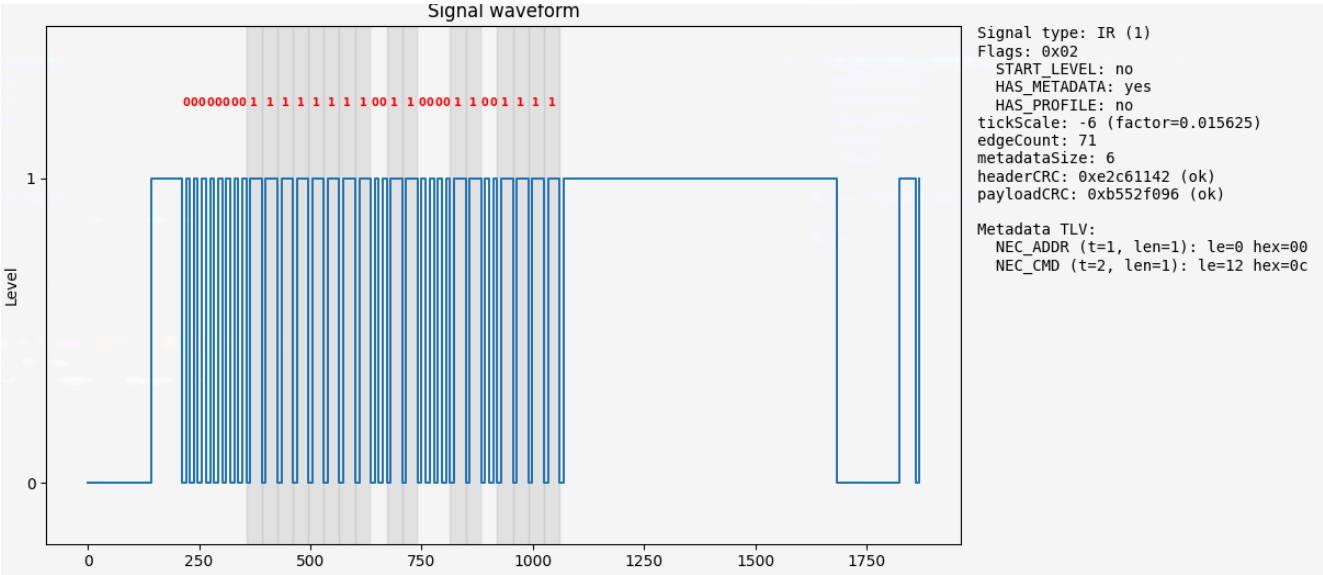


Figura 26. Análisis de un botón de un control remoto IR.

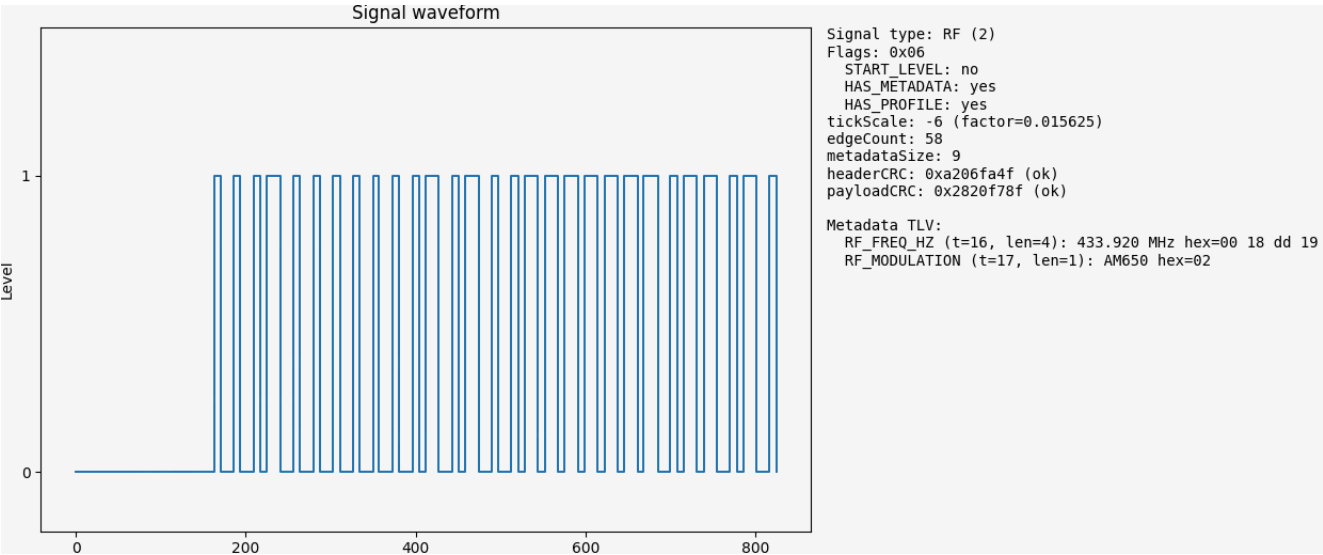


Figura 27. Análisis de un botón de un control remoto RF.

7. Conclusiones

A lo largo del semestre, se logró implementar un dispositivo que permite trabajar con señales de infrarrojos y radiofrecuencias. Se cumplieron todos los objetivos primarios: detectar y muestrear señales IR/RF, perdurarlas en una memoria microSD y la capacidad de realizarse desde el mismo dispositivo (es decir, la implementación de una interfaz física para el usuario — sin necesidad de un dispositivo externo para su funcionamiento).

Los objetivos secundarios no fueron realizados. Uno de ellos consta de interpretar protocolos comunes desde el mismo dispositivo — el mismo no pudo ser cumplido por falta de tiempo y pobre variedad de aparatos emisores para ensayar. Otro objetivo constaba de visualizar la forma de las señales con la pantalla de *Médium* — resultó fatuo para los gráficos más complejos debido al tamaño de pantalla limitante.

La realización del proyecto no ha presentado muchos inconvenientes. El principal de ellos fue la poca variedad de protocolos de RF para probar el módulo. Otro detalle que no se consideró en el diseño inicial es la ergonomía del dispositivo: podría fabricarse una caja de plástico (por ejemplo) donde ubicar el dispositivo de tal manera que el usuario pueda sujetarlo con ambas manos.

7.1. Cumplimiento

A continuación, se detallan los requerimientos planteados junto a

- para indicar su cumplimiento satisfactorio,
- para indicar su cumplimiento parcial o
- para indicar que no pudo ser cumplido.

De ser necesario, se adjunta alguna aclaración adicional.

- **Requerimientos de los periféricos (funcionales)**
 - El sistema debe contar con un transductor de señales IR ●.
 - El sistema debe ser capaz de demodular señales IR digitales ●.
 - El sistema debe ser capaz de transmitir señales IR ●.
 - El sistema debe contar con un transductor de señales de RF ●.
 - El módulo RF debe poder captar señales en bandas ISM/SRD comunes ● (433 MHz).
 - El módulo RF debe poder demodular señales digitales comunes (mínimamente OOK) ● (solo OOK).
 - El sistema debe ser capaz de transmitir señales RF ●.
 - El dispositivo debe contar con un puerto para conectar algún dispositivo de almacenamiento masivo como microSD ● (puerto para microSD).
 - El dispositivo debe contar con una pantalla para mostrar la interfaz de usuario ● (pantalla OLED).
 - El dispositivo debe contar botones (o similares) para que el usuario interactúe con el mismo ● (joystick).
- **Requerimientos del sistema (funcionales)**
 - El sistema debe contar con una interfaz de usuario para seleccionar distintos modos de funcionamiento ● (permite captar/emitir IR/RF).
 - La interfaz de usuario debe mostrarse en la pantalla ●.
 - La interfaz de usuario debe responder a los botones presionados por el usuario ●.
 - Cada periférico debe poder ser activado cuando el usuario lo necesite ●.
 - El dispositivo debe permitir almacenar las señales captadas en el sistema de almacenamiento ●.
 - El dispositivo debe reproducir señales almacenadas a través de los periféricos ●.
 - El sistema debe permitir eliminar señales almacenadas del sistema de almacenamiento ●.
 - El sistema debe registrar la fecha y hora de las capturas realizadas para su posterior trazabilidad ●.

- Este requerimiento se descartó porque se hubiera necesitado de un módulo extra que maneje fecha y hora. Se reemplazó por guardar la captura en uno de varios *slots* prefijados.
- **Requerimientos del proyecto (no funcionales)**
 - El proyecto debe desarrollarse utilizando el microcontrolador EDU-CIAA-NXP [2] como plataforma principal ● (única plataforma).
 - El código debe estar desarrollado en C/C++, compatible con los entornos de trabajo de la EDU-CIAA-NXP ●.
 - Todos los elementos físicos deben estar ensamblados en un único dispositivo final ●.
 - El diseño del dispositivo debe ser compacto y seguro para su manipulación ●.
 - El diseño del PCB fue pensado para ser lo más compacto posible al momento de encapsularse, pero no se llegó a diseñar una caja que contenga los circuitos.
 - La interfaz diseñada para la pantalla debe ser clara y sencilla, priorizando la facilidad de uso ●.
 - La transmisión y recepción de señales debe acatarse a las normativas del ENACOM vigentes a la fecha ●.
 - El almacenamiento de señales debe realizarse según estándares preexistentes ●.
 - Se descubrió que no existen estándares preexistentes almacenar señales digitales (ver Apéndice A.6).
 - La documentación del software debe incluir diagramas, descripciones de módulos y guías de uso para el usuario final ● (ver Apéndice A.5).
 - El proyecto debe estar finalizado, junto con su documentación correspondiente, antes del final de la cursada de “Taller de Proyecto I” del segundo semestre de 2025 ●.
 - El desarrollo del proyecto debe ajustarse a un presupuesto preestablecido ●.

7.2. División de tareas

Siguiendo con el tiempo dedicado al proyecto, no se ha desviado tanto del planteo original de la Tabla 8.

Tabla 8. División original de tareas

Integrante	Tareas
Juan Santiago	Integración del módulo microSD y desarrollo del sistema de archivos.
	Implementación de la interfaz de usuario: joystick y pantalla.
Lorenzo	Integración del módulo de Radiofrecuencia.
	Desarrollo de rutinas de captura y almacenamiento de tramas RF.
Gastón	Integración del módulo Infrarrojo (receptor y emisor).
	Desarrollo de rutinas de captura, análisis y reproducción de señales IR.
Todos	Implementación conjunta de aplicaciones IR y RF.
	Diseño de la PCB.
	Ensamblaje final del dispositivo y validación de funcionamiento

Juan terminó encargándose del driver de la pantalla OLED, del joystick y del diseño de las distintas pantallas, mientras que Santiago que dedicó al módulo microSD. Lorenzo y Gastón se ocuparon de los módulos RF e IR respectivamente.

El diseño del PCB fue realizado entre todos, pero con mayor impronta de Lorenzo y Gastón que ya contaban con experiencia previa de su paso por una secundaria técnica.

No se llevó un registro riguroso, pero se estiman 4 horas semanales aportadas al proyecto por cada integrante del grupo, considerando armado del PCB, realización de los informes, investigaciones y pruebas. A lo largo de 18 semanas, se tiene que cada integrante aportó 72 horas al proyecto, resultando en un total de 288 horas de ingeniería.

FECHA	AGOSTO		SEPTIEMBRE						OCTUBRE			NOVIEMBRE			DICIEMBRE				FEBRERO		
	18	25	1	8	15	22	29	6	16	20	3	10	17	1	11	15	18	2	5	9	
SEMANA:		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	
Elección de Grupo y Proyecto																					
Objetivos y Requerimientos																					
Diseño de Software																					
Investigación de periféricos																					
Programación de modulo IR																					
Programación de modulo RF																					
Programación de microSD																					
Pantalla lcd + teclado																					
Integración del código																					
Simulación y Pruebas																					
Diseño Esquemático																					
Diseño PCB																					
Fabricación																					
Ensamblaje																					
Ensayos																					
Informe/Muestra																					
Informe inicial de proyecto																					
Primer Informe de Avance																					
Segundo Informe de Avance																					
Horas semanales en Aula	4	4	4	4	4	4	4	4	4	4	4	4	4	4	2	4	4	4	4	4	
Total Horas en Aula/Taller		8	12	16	20	24	28	32	36	40	44	48	52	56	58	62	66	70	74	78	

Figura 28. Cronograma final realizado.

7.3. Conclusiones generales

Más allá del resultado final satisfactorio, se consideró muy valiosa la experiencia de trabajar en un proyecto un poco más ambicioso para lo que se exige en la carrera. Se aprendió a realizar estimaciones realistas, se claró con los objetivos y la forma de expresarse con los avances. Agradecemos a Juan Ignacio por guiarnos en este aspecto.

Por ello, consideramos que fue un proyecto más que enriquecedor para nuestra formación como futuros ingenieros.

Bibliografía

- [1] Flipper Devices Inc., «Flipper Zero». Accedido: 4 de septiembre de 2025. [En línea]. Disponible en: <https://docs.flipper.net/>
- [2] Proyecto CIAA, «EDU-CIAA-NXP». Accedido: 4 de septiembre de 2025. [En línea]. Disponible en: <https://www.proyecto-ciaa.com.ar/devwiki/doku.php%3Fid=desarrollo:edu-ciaa:edu-ciaa-nxp.html>
- [3] E. Pernia, «firmware_v3». [En línea]. Disponible en: https://github.com/epernia/firmware_v3
- [4] KiCad Developers, «KiCad». [En línea]. Disponible en: <https://www.kicad.org/>

A. Apéndices

A.1. Lista de materiales

En la [Tabla 9](#) se presenta la lista completa de insumos del proyecto, incluyendo los módulos y placas principales, componentes para regulación y alimentación, conectores, elementos de la botonera, y componentes específicos para la sección de IR. Esta lista detalla tanto la cantidad como las notas relevantes para cada componente, facilitando la adquisición y organización de los materiales necesarios para el desarrollo del proyecto.

Tabla 9. Insumos y componentes de Médium (*bill of materials*).

Cantidad	Componente	Notas
Módulos y placas principales		
1	EDU-CIAA-NXP	Placa base provista por la cátedra.
1	Módulo CC1101	Con interfaz SPI
1	Módulo microSD	Con interfaz SPI
1	OLED 1.3" 128×64	Monocromático con interfaz I ² C
1	Módulo IR TSOP38438	o equivalente TSOP382/384
1	LED IR emisor	Debe emitir a 940 nm
Regulación y alimentación		
1	Banco de pilas	pack/portapilas
2	Bornera 2 pines	Para entrada de pack de pilas
2	Condensador electrolítico	De 200 µF
1	Regulador 5V	LM7805
1	Diodo zener	1N4728
1	Capacitor	De 0,01 µF
1	Resistencia	De 10 ohm
2	Diodo rectificador	1N4007
1	Capacitor	De 0,01 µF
Conectores		
1	Conector hembra	Jack 2.1mm
2	Tiras de pines macho	20×2 (40 pines)
1	Tiras de pines hembra	20×1 (20 pines)
Botonera		
1	<i>Joystick</i>	O semejante ensamble de dos potenciómetros.
2	Pulsadores	O semejante ensamble de dos potenciómetros.
2	Condensador	De 100 nF, para antirrebote
2	Resistencia	De 10 kohm
Para IR		
1	Condensador	De 2,2 µF, para desacople V_{CC} del receptor.
1	Transistor	2N2222
2	Resistencias	de polarización

A.2. Presupuesto

En la [Tabla 10](#) se detallan los precios de los componentes utilizados para el desarrollo del proyecto. Específicamente, cuánto costó cada componente en el momento de su compra (en pesos argentinos, obtenidos entre agosto y diciembre de 2025). Además, se aproximó el costo de una hora de trabajo de ingeniería en USD 10 $\approx$ ARS 14.000.

Tabla 10. Insumos y componentes de Médium (*bill of materials*).

Cantidad	Componente	Presupuesto [ARS]
Módulos y placas principales		
1	EDU-CIAA-NXP	
1	Módulo CC1101	16.189
1	Módulo microSD	2.640
1	OLED 1.3" 128×64	8.990
1	Módulo IR TSOP38438	3.060
1	LED IR emisor	350
Regulación y alimentación		
1	Banco de pilas	6.600
2	Bornera 2 pines	540
2	Condensador electrolítico	800
1	Regulador 5V	1.500
1	Diodo zener	500
1	Capacitor	100
1	Resistencia	300
2	Diodo rectificador	500
Conectores		
1	Conector hembra	1.200
2	Tiras de pines macho	4.000
1	Tiras de pines hembra	1.000
Botonera		
1	<i>Joystick</i>	2.500
2	Pulsadores	400
2	Condensador	200
2	Resistencia	600
Para IR		
1	Condensador	400
1	Transistor	500
2	Resistencias	600
Precio de componentes		53.569\$
288 Horas trabajadas		4.050.000\$
Total		4.103.569\$

A.3. Circuitos y esquemáticos

En este apéndice se presentan los esquemáticos y circuitos desarrollados para el proyecto. Por un lado, en la [Figura 29](#) se muestra el esquemático completo del poncho realizado para integrar todos los componentes del proyecto. Además, en la [Tabla 11](#) se detallan los componentes utilizados, sus valores o tipos, y los footprints correspondientes para su montaje.

Tabla 11. Referencias de los componentes del poncho de Médium.

Ref.	Valor/tipo	Footprint
C1	2,2 μ F — electrolítico radial	Capacitor_THT: CP_Radial_D6.3mm_P2.50mm
C2	100 nF — cerámico disco	Capacitor_THT: C_Disc_D4.7mm_ W2.5mm_P5.00mm
C3		
C4	10 μ F — electrolítico radial	Capacitor_THT: CP_Radial_D6.3mm_P2.50mm
C5		
C6	100 nF — cerámico disco	Capacitor_THT: C_Disc_D4.7mm_ W2.5mm_P5.00mm
C7	100 nF - cerámico disco	Capacitor_THT: C_Disc_D3.4mm_ W2.1mm_P2.50mm
D1	SFH4550 — LED infrarrojo	LED_THT: LED_D5.0mm_IRGrey
D2	1N4007 - diodo rectificador	Diode_THT: D_D0-41_S0D81_ P10.16mm_Horizontal
D3	D_Zener — diodo zener	Diode_THT: D_D0-34_S0D68_ P10.16mm_Horizontal
D4	1N4007 - diodo rectificador	Diode_THT: D_D0-41_S0D81_ P10.16mm_Horizontal
JP1	Conectores EDU-CIAA — header doble 2x20	Connector_PinHeader_2.54mm: PinHeader_2x20_P2.54mm_Vertical
JP2		
J3	RF — conector RF	Connector_PinSocket_2.54mm: PinSocket_2x04_P2.54mm_Vertical
J4	Micro_SD - header 1x6	Connector_PinSocket_2.54mm: PinSocket_1x06_P2.54mm_Vertical
J5	OLED_128x64 - header 1x4	Connector_PinSocket_2.54mm: PinSocket_1x04_P2.54mm_Vertical
J6	P0T_X — header 1x3	Connector_PinSocket_2.54mm: PinSocket_1x03_P2.54mm_Vertical
J7	P0T_Y — header 1x3	
J10	Power_In - entrada de alimentación	Connector_PinHeader_2.54mm: PinHeader_1x02_P2.54mm_Vertical
J11		Power_5V - alimentación 5 V
Q1	2N2222 - transistor NPN	Package_T0_S0T_THT: T0-92L_Inline
R1	33 ohm — 1/4 W	Resistor_THT:R_Axial_DIN0207_L6. 3mm_D2.5mm_P10.16mm_Horizontal
R2	1 kohm — 1/4 W	
R3	100 kohm — 1/4 W	
R4		
R5	1 kohm — 1/4 W	
R13	100 kohm — 1/4 W	
SW1	Pulsador	Button_Switch_THT:SW_PUSH_6mm
SW2		
SW3		
SW7	Switch deslizable	Button_Switch_THT: SW_Slide_SPDT_Straight_ CK_0S102011MS2Q
U1	TSOP38338 — receptor IR	OptoDevice:Vishay_M0LD-3Pin
U2	LM7805 — regulador lineal	Package_T0_S0T_THT: T0-220-3_Vertical

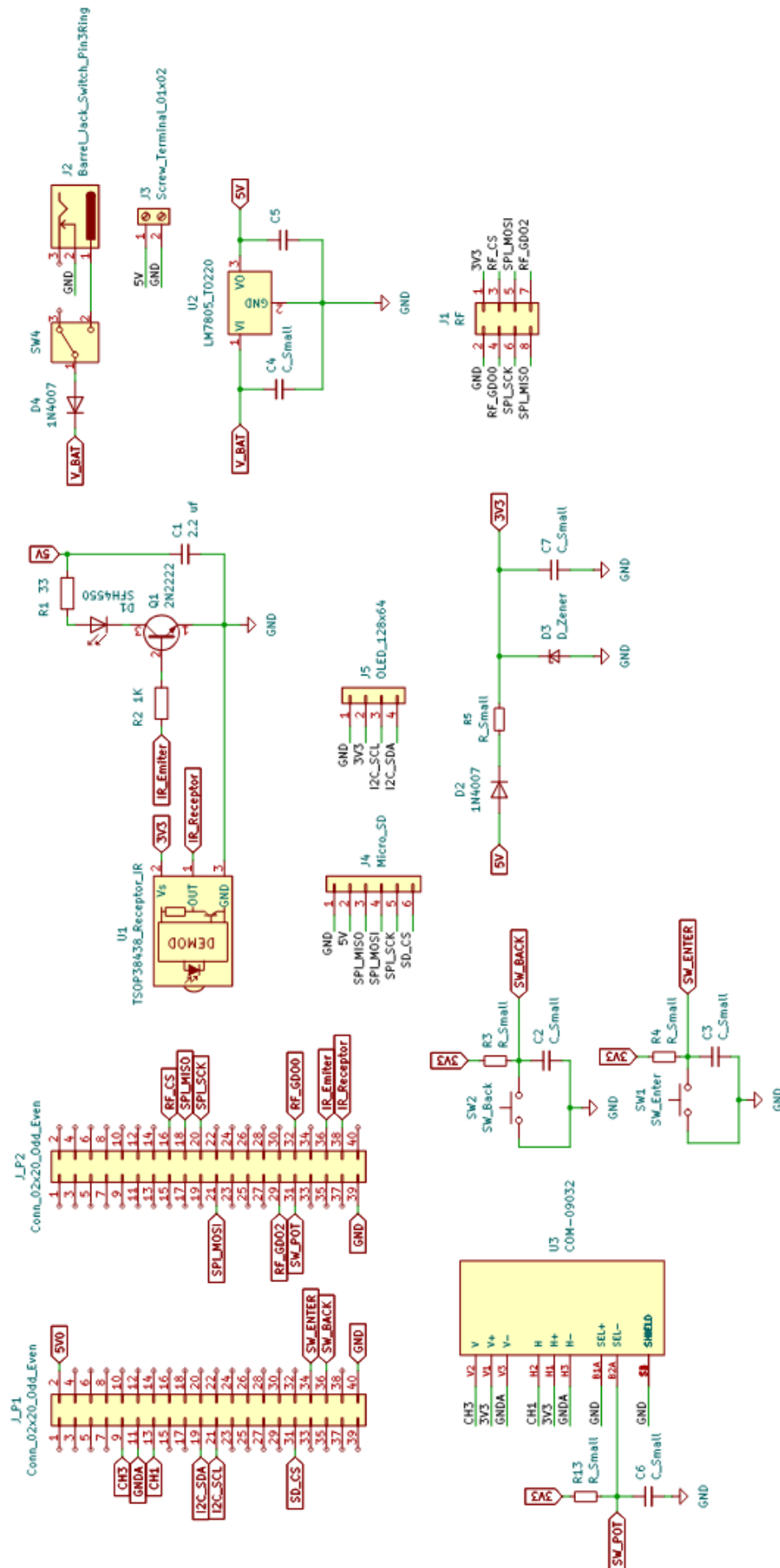


Figura 29. Esquemático completo del poncho de Medium.

A.4. PCB

En este apartado se muestra el diseño de la PCB del poncho de Médium, realizada con KiCad [4]. En la [Figura 30](#) se puede observar el diseño final de la PCB con los arreglos realizados, mientras que en la [Figura 31](#) se muestra el diseño originalmente impreso antes de realizar los arreglos.

Los componentes utilizados se detallan en el [Apéndice A.3](#). Más información sobre los arreglos realizados se encuentra en la [Sección 4.2](#).

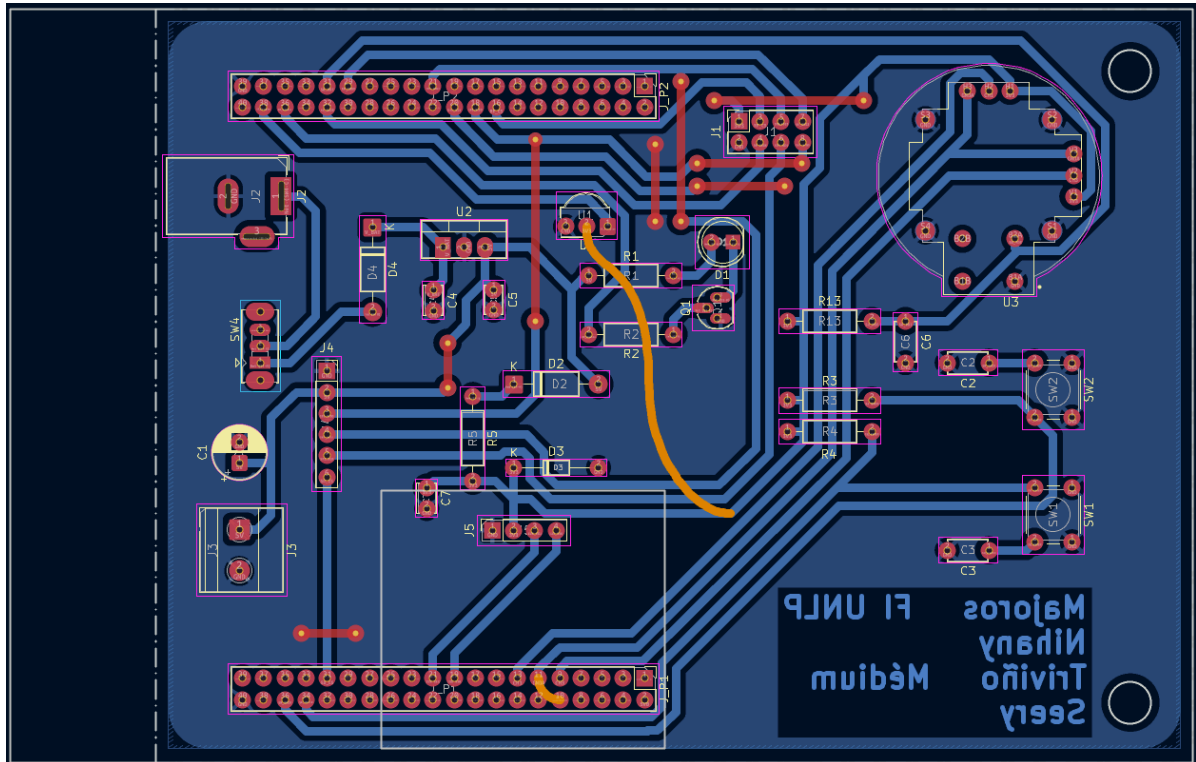


Figura 30. Diseño de la PCB del poncho de Médium con sus arreglos realizados.

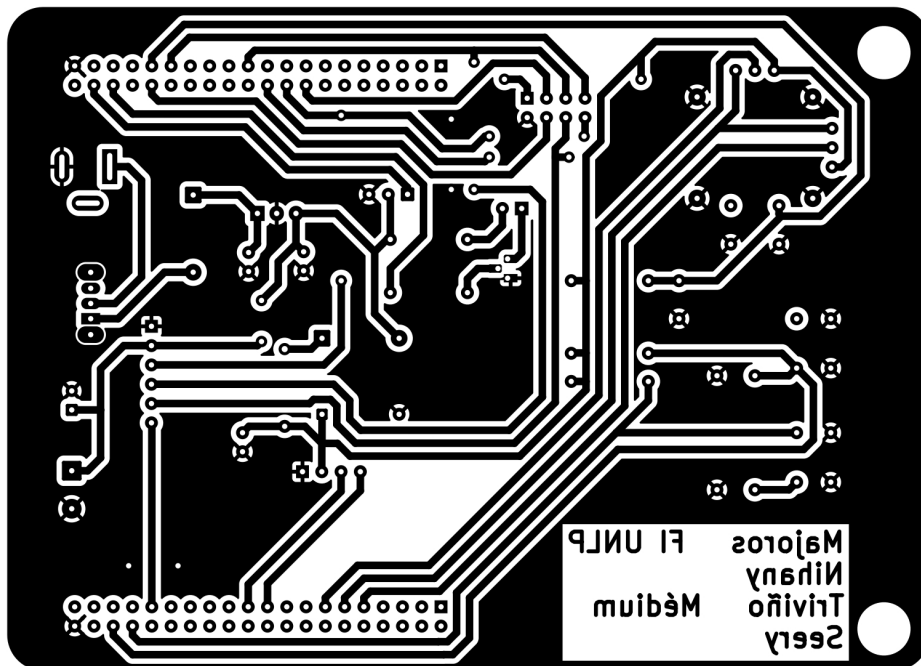


Figura 31. Diseño de la PCB originalmente impresa del poncho de Médium.

A.5. Manual de usuario

Para utilizar *Médium* se requiere:

- dos pilas AA cargadas (o la alimentación definida para la placa),
- tarjeta microSD insertada y formateada en FAT32.

Al encender, si la microSD no está disponible, el equipo muestra el mensaje *Inserta la microSD* y no habilita las acciones de captura/replay.

A.5.1. Controles del equipo

La interacción se realiza con:

- *joystick* (ejes X/Y): navegación por menús y acciones,
- botón ENTER: confirmar opción,
- botón BACK: volver/cancelar.

La pantalla principal permite elegir modo: **IR** o **RF**. Luego se ingresa al menú de *slots* del modo seleccionado.

A.5.2. Organización de archivos en microSD

Médium trabaja con slots fijos:

- IR1.sig a IR5.sig para infrarrojo,
- RF1.sig a RF5.sig para radiofrecuencia.

Cada slot puede estar vacío (acción disponible: grabar) u ocupado (acciones disponibles: reproducir o borrar).

A.5.3. Flujo de uso en el dispositivo

A.5.3.1. Grabar una señal IR

- 1) En la pantalla principal, seleccionar **IR** y confirmar.
- 2) Elegir un slot vacío.
- 3) Seleccionar la acción de *grabar*.
- 4) Apuntar al emisor IR hacia el receptor y emitir la señal.
- 5) Para cancelar, presionar **BACK**.

Si la captura es válida y la SD está disponible, el archivo se guarda en el slot elegido.

A.5.3.2. Grabar una señal RF (CC1101)

- 1) En la pantalla principal, seleccionar **RF** y confirmar.
- 2) Elegir un slot vacío y luego la acción de *grabar*.
- 3) Configurar los parámetros de captura:
 - Frecuencia: 433,92 MHz o 315 MHz.
 - Modulación: **AM650** o **AM270**.
- 4) Confirmar y emitir la señal RF a capturar.
- 5) Para cancelar, presionar **BACK**.

Durante el guardado se almacena, además de la forma de onda, la metadata de RF (frecuencia y modulación), que luego se reutiliza en la reproducción.

A.5.3.3. Reproducir una señal

- 1) Seleccionar modo (**IR** o **RF**).
- 2) Ingresar al listado de slots.
- 3) Elegir un slot ocupado.
- 4) Seleccionar acción *play*.

En RF, si el .sig contiene metadata, el equipo reaplica automáticamente la frecuencia y la modulación guardadas antes de emitir.

A.5.3.4. Borrar una señal

- 1) Seleccionar modo.
- 2) Elegir slot ocupado.
- 3) Seleccionar acción *delete*.

Esta operación elimina el archivo .sig del slot y no se puede deshacer.

A.5.4. Recomendaciones de uso

- Mantener la microSD insertada durante toda la operación.
- En IR, ubicar emisor y receptor enfrentados y a corta distancia para mejorar la captura.
- En RF, evitar fuentes de ruido cercanas y respetar la frecuencia esperada del dispositivo objetivo.

- Si una captura falla, repetirla ajustando distancia/orientación o configuración RF.

A.5.5. Scripts de PC para desarrollo y análisis

En scripts/custom se incluyen herramientas para trabajar con el dispositivo desde una PC.

A.5.5.1. Preparación del entorno

- 1) Instalar uv.
- 2) Ejecutar:
`cd scripts/custom`
`uv sync`

A.5.5.2. Terminal serie (**terminal.py**)

Permite ver logs UART y enviar texto a la placa.

```
cd scripts/custom
uv run terminal.py --list-ports
uv run terminal.py
Opcionalmente se puede fijar puerto/baudrate:
uv run terminal.py /dev/ttyUSB1 --baudrate 115200
```

A.5.5.3. Inspección de archivos **.sig** (**signals.py**)

Permite validar CRC, parsear metadata y visualizar la forma de onda.

```
cd scripts/custom
uv run signals.py RF1.sig
Para exportar imagen sin abrir ventana interactiva:
uv run signals.py RF1.sig --save RF1.png
```

A.5.6. Solución rápida de problemas

- *Mensaje “Insertá la microSD” permanente*: verificar formato FAT32, contactos de la tarjeta e inserción completa.
- *No guarda en slot*: revisar que la SD siga montada y que el slot seleccionado corresponda al modo actual.
- *No reproduce en RF*: confirmar frecuencia/modulación esperada del receptor objetivo.
- *Sin logs UART en PC*: revisar puerto serie, permisos de sistema y baudrate 115.200 Hz.

A.6. Formato de archivo .sig

El dispositivo guarda las señales capturadas en archivos con extensión **.sig**, utilizando un formato binario propio orientado a:

- almacenar secuencias temporales de IR y RF en poco espacio,
- mantener compatibilidad entre captura y replay,
- incorporar verificación de integridad con CRC.

La implementación de referencia está en `program/src/utils/sig.c` y su interfaz en `program/inc/utils/sig.h`.

A.6.1. Estructura general

Cada archivo se compone de:

- 1) Header fijo de 32 bytes.
- 2) Bloque de duraciones (*edges*) codificado en longitud variable.
- 3) Bloque opcional de metadata en formato TLV.

A.6.1.1. Header (32 bytes)

Tabla 12. Campos del header del formato .sig.

Offset	Campo	Descripción
0..3	magic	SIG1
4	signalType	Tipo de señal: 1 IR, 2 RF.
5	flags	Bits de estado (START_LEVEL, HAS_METADATA, HAS_PROFILE).
6	tickScale	Escala temporal (en esta versión se usa -6, equivalente a 1 μ s por tick).
7	Reservado	Actualmente en 0.
8..11	edgeCount	Cantidad de duraciones serializadas.
12..15	dataOffset	Inicio del bloque de <i>edges</i> (debe ser 32).
16..19	metaOffset	Inicio de metadata (0 si no hay metadata).
20..23	metaSize	Tamaño total de metadata en bytes.
24..27	payloadCrc32	CRC32 de <i>edges</i> codificados + metadata.
28..31	headerCrc32	CRC32 del header con este campo forzado a 0 para el cálculo.

A.6.1.2. Codificación de duraciones (*edges*)

Cada duración se serializa en little-endian con dos variantes:

- Si `ticks <= 0xFFFFE`: se guarda en 2 bytes (`uint16`).
- Si `ticks > 0xFFFFE`: se usa secuencia extendida de 6 bytes: marcador `0xFFFF` + valor real en `uint32`.

Esta estrategia reduce tamaño para pulsos cortos, manteniendo soporte para duraciones largas cuando aparecen.

A.6.1.3. Metadata opcional (TLV)

Cuando existe metadata, se guarda al final del payload como secuencia TLV (*type*, *len*, *value*), donde:

- *type*: 1 byte.
- *len*: 1 byte.
- *value*: *len* bytes.

Tipos utilizados actualmente:

- 1 (SIG_META_NEC_ADDR): dirección NEC (1 byte).
- 2 (SIG_META_NEC_CMD): comando NEC (1 byte).
- 16 (SIG_META_RF_FREQ_HZ): frecuencia RF en Hz (4 bytes, little-endian).
- 17 (SIG_META_RF_MODULATION): modulación RF (1 byte: AM270 o AM650).

A.6.1.4. Verificación de integridad

El formato usa CRC32 IEEE (polinomio `0xEDB88320`) en dos niveles:

- `headerCrc32`: protege el header completo.
- `payloadCrc32`: protege el bloque de datos (*edges* codificados + metadata).

Durante la lectura se verifica primero el header y luego el payload. Si alguno falla, el archivo se descarta.

A.6.2. Script de inspección

El script `scripts/custom/signals.py` permite analizar archivos `.sig` fuera del firmware. Entre otras funciones:

- valida `headerCrc32` y `payloadCrc32`,
- parsea metadata TLV y decodifica campos conocidos,
- reconstruye y grafica la forma de onda temporal para inspección visual.